LAPORAN PRAKTIKUM TUGAS AKHIR PENGEMBANGAN APLIKASI BERBASIS WEB

Modul 8: HMVC, Authorization,
dan AJAX pada Laravel



Disusun Oleh:

Nama: Dryhus Dzacky Damingtyas.S
NIM: 707012530010
Kelas: D4SIKC-49-04
Mata Kuliah: Pengembangan Aplikasi Berbasis Web

UNIVERSITAS TELKOM
FAKULTAS ILMU TERAPAN
D4 SISTEM INFORMASI KOTACERDAS

A. Tujuan

Lewat praktikum ini saya membangun SiPustaka, aplikasi perpustakaan kampus, dari project Laravel yang masih kosong dengan arsitektur HMVC. Admin mengelola buku dan akun pengguna, sedangkan user hanya melihat daftar buku dan mencatat peminjaman. Hal yang ingin saya capai adalah sebagai berikut.

1. Memasang nwidart/laravel-modules pada Laravel 12 dan mengatur merge-plugin di composer.json supaya kelas dalam folder Modules dikenali.

2. Membagi aplikasi menjadi tiga modul, yaitu Buku, Peminjaman, dan UserManagement, masing-masing dengan controller, model, request, route, migration, dan view sendiri.

3. Membuat register, login, dan logout secara manual di luar modul, lalu menambahkan kolom role pada tabel users beserta seeder admin dan user.

4. Membuat middleware role dan memastikan akun hasil register selalu berperan user.

5. Membuat CRUD Buku secara synchronous lengkap dengan pencarian dan pagination, dengan hak ubah data hanya untuk admin.

6. Membuat CRUD Peminjaman dengan AJAX tanpa memuat ulang halaman, termasuk upload berkas yang bisa dibuka lagi, diganti saat edit, dan ikut terhapus dari storage.

7. Membuat CRUD user untuk admin dengan batasan bahwa admin tidak bisa menghapus atau menurunkan role akunnya sendiri.

8. Menampilkan pesan error di bawah setiap input, termasuk error 422 pada form AJAX, dan membuktikan pembatasan akses lewat error 403.

B. Alat dan Bahan

Praktikum dikerjakan pada laptop dengan spesifikasi dan perangkat lunak berikut.

Perangkat Keras

● Prosesor 13th Gen Intel Core i7-13620H.

● RAM 16 GB (terbaca 15,7 GB).

● Windows 11 Home Single Language build 10.0.26200.

Perangkat Lunak

● XAMPP dengan Apache 2.4.58, MariaDB 10.4.32, PHP 8.2.12, dan phpMyAdmin 5.2.1.

● Composer 2.10.3.

● Laravel Framework 12.70.2.

● nwidart/laravel-modules 12.0.5.

● Bootstrap 5.3.0, Bootstrap Icons 1.11.3, dan jQuery 3.6.0, semuanya dimuat lewat CDN.

● PHPUnit 11.5.57 untuk pengujian otomatis.

● Visual Studio Code 1.144.0.

● Google Chrome 156.0.8078.80 beserta DevTools.

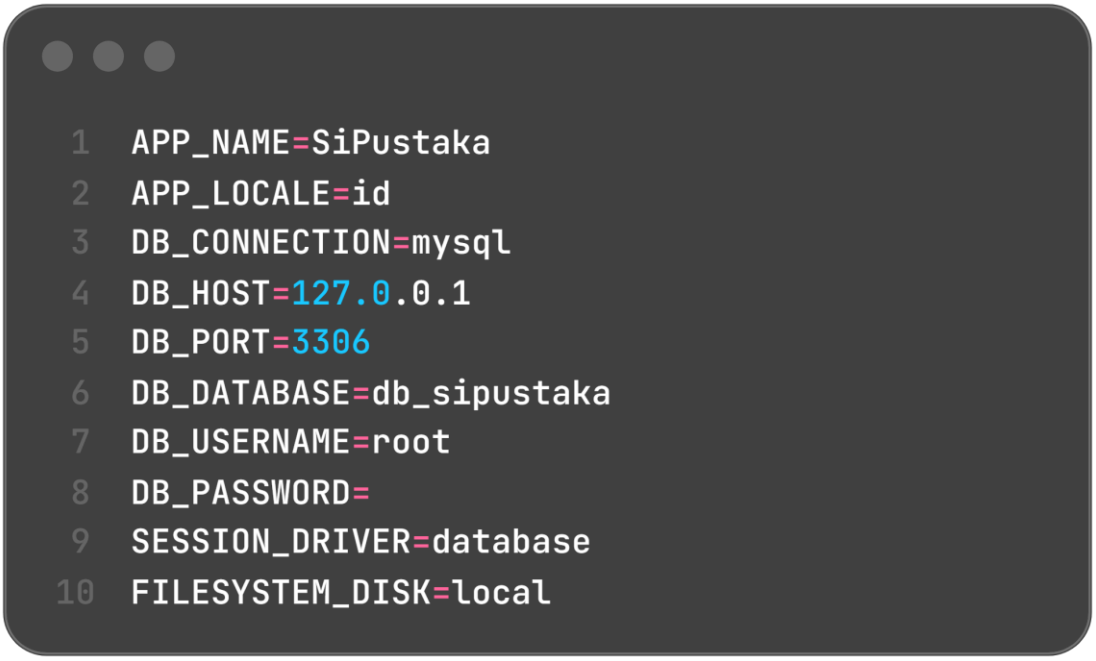
● Git 2.53.0.windows.1.

C. Langkah-Langkah Praktikum

Project saya beri nama sipustaka_hmvc dan memakai database db_sipustaka di MariaDB. Autentikasi saya tulis sendiri di folder app, sedangkan fitur buku, peminjaman, dan pengelolaan user masing-masing menjadi satu modul.

C.1 Menyiapkan Project, Database, dan HMVC

Koneksi database diatur di file .env. APP_LOCALE saya ubah ke id supaya pesan bawaan Laravel memakai bahasa Indonesia. Session disimpan di database, sedangkan disk bawaan tetap local karena berkas upload nanti memakai disk public secara eksplisit.



Kode 1. Pengaturan aplikasi dan database pada file .env.

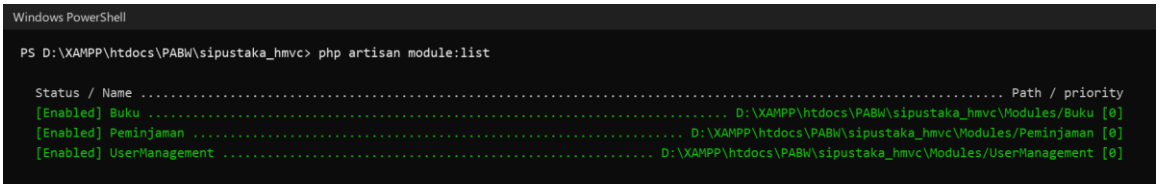
Paket HMVC dipasang dengan composer require nwidart/laravel-modules. Setelah itu composer.json saya beri blok merge-plugin. Blok ini membuat Composer ikut membaca composer.json milik tiap modul, jadi namespace Modules\... bisa di-autoload tanpa menambah aturan PSR-4 satu per satu.

```json
"require": {
    "php": "^8.2",
    "laravel/framework": "^12.0",
    "laravel/tinker": "^2.10.1",
    "nwidart/laravel-modules": "^12.0"
},

"extra": {
    "laravel": {
        "dont-discover": []
    },
    "merge-plugin": {
        "include": [
            "Modules/*/composer.json"
        ]
    }
}
```

Kode 2. Bagian require dan extra merge-plugin pada composer.json.

Tiga modul dibuat dengan perintah php artisan module:make Buku, module:make Peminjaman, dan module:make UserManagement. Ketiganya tampil Enabled pada module:list.



Gambar 1. Hasil php artisan module:list.

C.2 Kolom Role, Enum, Model User, dan Seeder

Kolom role ditambahkan lewat migration baru, tidak dengan mengubah migration users bawaan. Nilai bawaannya user, sehingga akun tanpa keterangan role otomatis menjadi user biasa.

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->string('role')->default('user')->after('email');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn('role');
        });
    }
};
```

Kode 3. Migration penambah kolom role pada tabel users.

Dua nilai role dibungkus dalam backed enum UserRole. Dengan enum, aturan validasi cukup memakai Rule::enum dan nilai di luar admin atau user langsung ditolak.

```php
<?php

namespace App\Enums;

enum UserRole: string
{
    case Admin = 'admin';
    case User = 'user';

    /**
     * Get the human readable label of the role.
     */
    public function label(): string
    {
        return match ($this) {
            self::Admin => 'Admin',
            self::User => 'User',
        };
    }
}
```

Kode 4. Enum UserRole.

Model User meng-cast kolom role ke enum dan punya nilai bawaan user. Method isAdmin() dipakai di view untuk menyembunyikan menu, sedangkan hasRole() menerima satu atau beberapa role dan dipakai oleh middleware.



```php
<?php

namespace App\Models;

// use Illuminate\Contracts\Auth\MustVerifyEmail;
use App\Enums\UserRole;
use Database\Factories\UserFactory;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
    /**
     * The model's default attribute values.
     *
     * @var array<string, mixed>
     */
    protected $attributes = [
        'role' => 'user',
    ];

    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
            'role' => UserRole::class,
        ];
    }

    /**
     * Determine whether the user has the admin role.
     */
    public function isAdmin(): bool
    {
        return $this->role === UserRole::Admin;
    }

    /**
     * Determine whether the user has one of the given roles.
     */
    public function hasRole(UserRole|string ...$roles): bool
    {
        foreach ($roles as $role) {
            if ($this->role === ($role instanceof UserRole ? $role :
UserRole::tryFrom($role))) {
                return true;
            }
        }

        return false;
    }
}
```

Kode 5. Model User dengan cast role, isAdmin(), dan hasRole().

Seeder membuat dua akun, yaitu Administrator (admin@example.com) dan User Biasa (user@example.com). Keduanya memakai password password. State admin() pada UserFactory menentukan role akun pertama.

```php
<?php

namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    use WithoutModelEvents;

    /**
     * Seed the application's database.
     */
    public function run(): void
    {
        User::factory()->admin()->create([
            'name' => 'Administrator',
            'email' => 'admin@example.com',
            'password' => 'password',
        ]);

        User::factory()->create([
            'name' => 'User Biasa',
            'email' => 'user@example.com',
            'password' => 'password',
        ]);
    }
}
```

Kode 6. DatabaseSeeder dengan satu akun admin dan satu akun user.

C.3 Autentikasi di Luar Modul

Register, login, dan logout saya letakkan di app/Http/Controllers/Auth. Autentikasi dipakai semua modul, jadi tempatnya lebih cocok di luar salah satu modul. Route login dan register dibungkus middleware guest, sedangkan dashboard dan logout dibungkus auth.

```php
<?php

use App\Http\Controllers\Auth\LoginController;
use App\Http\Controllers\Auth\RegisterController;
use App\Http\Controllers\DashboardController;
use Illuminate\Support\Facades\Route;

Route::redirect('/', '/dashboard');

Route::middleware('guest')->group(function () {
    Route::get('login', [LoginController::class, 'create'])->name('login');
    Route::post('login', [LoginController::class, 'store'])->name('login.store');
    Route::get('register', [RegisterController::class, 'create'])->name('register');
    Route::post('register', [RegisterController::class, 'store'])->name('register.store');

});

Route::middleware('auth')->group(function () {
    Route::get('dashboard', DashboardController::class)->name('dashboard');
    Route::post('logout', [LoginController::class, 'destroy'])->name('logout');
});
```

Kode 7. Route autentikasi dan dashboard pada routes/web.php.

LoginController membatasi percobaan login dengan RateLimiter. Kuncinya gabungan email dan IP, dan setelah lima kali gagal permintaan berikutnya ditolak beserta sisa waktu tunggu. Setelah login berhasil session di-regenerate, lalu pengguna diarahkan ke halaman yang tadinya dituju.

```php
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Http\Requests\Auth\LoginRequest;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\RateLimiter;
use Illuminate\Support\Str;
use Illuminate\Validation\ValidationException;
use Illuminate\View\View;

class LoginController extends Controller
{
    /**
     * Authenticate the user.
     */
    public function store(LoginRequest $request): RedirectResponse
    {
        $throttleKey = Str::transliterate(Str::lower($request->string('email')).'|'.$request->ip());

        if (RateLimiter::tooManyAttempts($throttleKey, 5)) {
            throw ValidationException::withMessages([
                'email' => 'Terlalu banyak percobaan login. Coba lagi dalam '.RateLimiter::availableIn($throttleKey).' detik.',
            ]);
        }

        if (! Auth::attempt($request->validated(), $request->boolean('remember'))) {
            RateLimiter::hit($throttleKey);

            throw ValidationException::withMessages([
                'email' => 'Email atau password salah.',
            ]);
        }

        RateLimiter::clear($throttleKey);
        $request->session()->regenerate();

        return redirect()->intended(route('dashboard'));
    }

    /**
     * Log the user out.
     */
    public function destroy(Request $request): RedirectResponse
    {
        Auth::logout();

        $request->session()->invalidate();
        $request->session()->regenerateToken();

        return redirect()->route('login');
    }
}
```

Kode 8. LoginController dengan rate limiter.

RegisterController hanya mengambil name, email, dan password dari request, lalu role diisi UserRole::User di server. Dengan begitu field role yang diselipkan ke request tidak berpengaruh dan akun hasil register selalu menjadi user.

```php
<?php

namespace App\Http\Controllers\Auth;

use App\Enums\UserRole;
use App\Http\Controllers\Controller;
use App\Http\Requests\Auth\RegisterRequest;
use App\Models\User;
use Illuminate\Http\RedirectResponse;
use Illuminate\Support\Facades\Auth;
use Illuminate\View\View;

class RegisterController extends Controller
{
    /**
     * Show the registration form.
     */
    public function create(): View
    {
        return view('auth.register');
    }

    /**
     * Register a new user. Self-registered accounts always get the "user" role.
     */
    public function store(RegisterRequest $request): RedirectResponse
    {
        $user = User::create([
            ...$request->safe()->only(['name', 'email', 'password']),
            'role' => UserRole::User,
        ]);

        Auth::login($user);
        $request->session()->regenerate();

        return redirect()->route('dashboard')->with('success', 'Registrasi berhasil. Selamat datang!');
    }
}
```

Kode 9. RegisterController yang selalu memberi role user.

C.4 Middleware Role

Middleware EnsureUserHasRole menerima daftar role sebagai parameter, misalnya role:admin atau role:admin,user. Jika pengguna belum login atau rolenya tidak ada dalam daftar, permintaan dihentikan dengan abort(403).

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class EnsureUserHasRole
{
    /**
     * Ensure the authenticated user has one of the given roles.
     * Usage: ->middleware('role:admin') or ->middleware('role:admin,user')
     *
     * @param  Closure(Request): (Response)  $next
     */
    public function handle(Request $request, Closure $next, string ...$roles): Response
    {
        $user = $request->user();

        if ($user === null || ! $user->hasRole(...$roles)) {
            abort(403, 'Anda tidak memiliki hak akses ke halaman ini.');
        }

        return $next($request);
    }
}
```

Kode 10. Middleware EnsureUserHasRole.

Alias role didaftarkan di bootstrap/app.php. Di file yang sama saya mengatur tujuan redirect untuk tamu (ke halaman login) dan untuk pengguna yang sudah login (ke dashboard).

```php
<?php

use App\Http\Middleware\EnsureUserHasRole;
use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware): void {
        $middleware->alias([
            'role' => EnsureUserHasRole::class,
        ]);

        $middleware->redirectGuestsTo(fn () => route('login'));
        $middleware->redirectUsersTo(fn () => route('dashboard'));
    })
    ->withExceptions(function (Exceptions $exceptions): void {
        //
    })->create();
```

Kode 11. Alias middleware role dan pengaturan redirect di bootstrap/app.php.

C.5 Modul Buku (CRUD Synchronous)

Daftar buku boleh dilihat admin dan user, tetapi route create, store, edit, update, dan destroy hanya berada dalam grup role:admin. Karena itu user yang mengetik alamat buku/create langsung mendapat 403.

```php
<?php

use Illuminate\Support\Facades\Route;
use Modules\Buku\Http\Controllers\BukuController;

Route::middleware(['auth', 'role:admin,user'])->group(function () {
    Route::get('buku', [BukuController::class, 'index'])->name('buku.index');
});

Route::middleware(['auth', 'role:admin'])->group(function () {
    Route::get('buku/create', [BukuController::class, 'create'])->name('buku.create');
    Route::post('buku', [BukuController::class, 'store'])->name('buku.store');
    Route::get('buku/{buku}/edit', [BukuController::class, 'edit'])->name('buku.edit');
    Route::put('buku/{buku}', [BukuController::class, 'update'])->name('buku.update');
    Route::delete('buku/{buku}', [BukuController::class, 'destroy'])-
>name('buku.destroy');
});
```

Kode 12. Route modul Buku.

Tabel buku memiliki lima kolom data, yaitu judul, penulis, penerbit, tahun_terbit, dan stok. Aturan validasinya ada pada BukuRequest, termasuk batas tahun terbit yang tidak boleh melewati tahun sekarang dan stok yang tidak boleh negatif.

```php
<?php

namespace Modules\Buku\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class BukuRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        return [
            'judul' => ['required', 'string', 'max:255'],
            'penulis' => ['required', 'string', 'max:255'],
            'penerbit' => ['required', 'string', 'max:255'],
            'tahun_terbit' => ['required', 'integer', 'digits:4', 'min:1900', 'max:'.(int) date('Y')],
            'stok' => ['required', 'integer', 'min:0'],
        ];
    }

    /**
     * Get custom messages for validator errors.
     *
     * @return array<string, string>
     */
    public function messages(): array
    {
        return [
            'tahun_terbit.digits' => 'Tahun terbit harus 4 digit angka.',
            'tahun_terbit.max' => 'Tahun terbit tidak boleh melebihi tahun sekarang.',
            'stok.min' => 'Stok tidak boleh bernilai negatif.',
        ];
    }

    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return $this->user()?->hasRole('admin', 'user') ?? false;
    }
}
```

Kode 13. BukuRequest dengan pesan error berbahasa Indonesia.

Method index menerima kata kunci pencarian pada judul, penulis, atau penerbit, lalu menampilkan sepuluh buku per halaman. withQueryString() menjaga kata kunci tetap ada saat pindah halaman.

```php
<?php

namespace Modules\Buku\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\View\View;
use Modules\Buku\Http\Requests\BukuRequest;
use Modules\Buku\Models\Buku;

class BukuController extends Controller
{
    /**
     * Display a listing of books, optionally filtered by keyword.
     */
    public function index(Request $request): View
    {
        $search = $request->string('search')->trim()->toString();

        $bukus = Buku::query()
            ->when($search !== '', function (Builder $query) use ($search) {
                $query->where(function (Builder $query) use ($search) {
                    $query->where('judul', 'like', "%{$search}%")
                        ->orWhere('penulis', 'like', "%{$search}%")
                        ->orWhere('penerbit', 'like', "%{$search}%");
                });
            })
            ->orderBy('judul')
            ->paginate(10)
            ->withQueryString();

        return view('buku::index', [
            'bukus' => $bukus,
            'search' => $search,
        ]);
    }
}
```

Kode 14. BukuController, method index dengan pencarian dan pagination.

Method create, store, edit, update, dan destroy memakai redirect dengan pesan session, sehingga halaman dimuat ulang setiap kali ada aksi.

```php
class BukuController extends Controller
{
    /**
     * Show the form for creating a new book.
     */
    public function create(): View
    {
        return view('buku::create');
    }

    /**
     * Store a newly created book.
     */
    public function store(BukuRequest $request): RedirectResponse
    {
        Buku::create($request->validated());

        return redirect()->route('buku.index')->with('success', 'Buku berhasil ditambahkan.');
    }

    /**
     * Show the form for editing the specified book.
     */
    public function edit(Buku $buku): View
    {
        return view('buku::edit', ['buku' => $buku]);
    }

    /**
     * Update the specified book.
     */
    public function update(BukuRequest $request, Buku $buku): RedirectResponse
    {
        $buku->update($request->validated());

        return redirect()->route('buku.index')->with('success', 'Buku berhasil diperbarui.');
    }

    /**
     * Remove the specified book.
     */
    public function destroy(Buku $buku): RedirectResponse
    {
        $buku->delete();

        return redirect()->route('buku.index')->with('success', 'Buku berhasil dihapus.');

    }
}
```

Kode 15. BukuController, method penambah, pengubah, dan penghapus data.

Kolom Aksi pada view hanya dirender jika auth()->user()->isAdmin() bernilai true. Penyembunyian tombol ini hanya soal tampilan, sedangkan pembatasan yang sebenarnya tetap dilakukan middleware pada route.

```blade
<div class="table-responsive">
    <table class="table table-hover">
        <thead>
            <tr>
                <th width="5%">No</th>
                <th>Judul</th>
                <th>Penulis</th>
                <th>Penerbit</th>
                <th>Tahun</th>
                <th>Stok</th>
                @if (auth()->user()->isAdmin())
                    <th class="text-end">Aksi</th>
                @endif
            </tr>
        </thead>
        <tbody>
            @forelse ($bukus as $buku)
                <tr>
                    <td class="text-muted">{{ $bukus->firstItem() + $loop->index }}</td>
                    <td class="fw-medium">{{ $buku->judul }}</td>
                    <td>{{ $buku->penulis }}</td>
                    <td>{{ $buku->penerbit }}</td>
                    <td>{{ $buku->tahun_terbit }}</td>
                    <td>
                        <span class="badge {{ $buku->stok > 0 ? 'text-bg-success' : 'text-bg-secondary' }}">{{ $buku->stok }}</span>
                    </td>
                    @if (auth()->user()->isAdmin())
                        <td class="text-end text-nowrap">
                            <a href="{{ route('buku.edit', $buku) }}" class="btn btn-sm btn-outline-primary">
                                <i class="bi bi-pencil"></i> Edit
                            </a>
                            <form action="{{ route('buku.destroy', $buku) }}" method="POST" class="d-inline" data-confirm="Buku &quot;{{ $buku->judul }}&quot; akan dihapus dan tidak bisa dikembalikan.">
                                @csrf
                                @method('DELETE')
                                <button type="submit" class="btn btn-sm btn-outline-danger">
                                    <i class="bi bi-trash"></i> Hapus
                                </button>
                            </form>
                        </td>
                    @endif
                </tr>
            @empty
                <tr>
                    <td colspan="{{ auth()->user()->isAdmin() ? 7 : 6 }}">
                        <div class="empty-state">
                            <i class="bi bi-book"></i>
                            <p class="mt-2 mb-0">Tidak ada buku yang cocok dengan pencarian.</p>
                        </div>
                    </td>
                </tr>
            @endforelse
        </tbody>
    </table>
</div>
```

Kode 16. Tabel pada view index modul Buku.

C.6 Modul Peminjaman (CRUD AJAX)

Tabel peminjamen dibuat lewat dua migration. Migration pertama berisi lima kolom data dengan status Dipinjam atau Dikembalikan. Kolom berkas dan nama_berkas ditambahkan oleh migration kedua karena fitur upload baru dikerjakan setelah tabel awal jadi.

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('peminjamen', function (Blueprint $table) {
            $table->id();
            $table->string('nama_peminjam');
            $table->string('judul_buku');
            $table->date('tanggal_pinjam');
            $table->date('tanggal_kembali')->nullable();
            $table->enum('status', ['Dipinjam', 'Dikembalikan'])->default('Dipinjam');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('peminjamen');
    }
};
```

Kode 17. Migration pembuat tabel peminjamen.

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('peminjamen', function (Blueprint $table) {
            $table->string('berkas')->nullable()->after('status');
            $table->string('nama_berkas')->nullable()->after('berkas');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('peminjamen', function (Blueprint $table) {
            $table->dropColumn(['berkas', 'nama_berkas']);
        });
    }
};
```

Kode 18. Migration penambah kolom berkas dan nama_berkas.

Di tabel hanya tersimpan path berkas dan nama aslinya. Accessor berkas_url mengubah path itu menjadi URL lengkap yang ikut terkirim dalam JSON, jadi JavaScript tidak perlu menyusun alamat sendiri.

```php
<?php

namespace Modules\Peminjaman\Models;

use Illuminate\Database\Eloquent\Casts\Attribute;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Modules\Peminjaman\Database\Factories\PeminjamanFactory;

class Peminjaman extends Model
{
    /**
     * The attributes that are mass assignable.
     *
     * @var list<string>
     */
    protected $fillable = [
        'nama_peminjam',
        'judul_buku',
        'tanggal_pinjam',
        'tanggal_kembali',
        'status',
        'berkas',
        'nama_berkas',
    ];

    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'tanggal_pinjam' => 'date:Y-m-d',
            'tanggal_kembali' => 'date:Y-m-d',
        ];
    }

    /**
     * Public URL of the uploaded file, or null when there is no file.
     */
    protected function berkasUrl(): Attribute
    {
        return Attribute::get(fn (): ?string => $this->berkas ? asset('storage/'.$this->berkas) : null);
    }
}
```

Kode 19. Model Peminjaman dengan accessor berkas_url.

PeminjamanRequest membedakan aturan berkas untuk tambah dan ubah. Saat tambah berkas wajib ada, saat ubah boleh kosong agar berkas lama dipertahankan. Format yang diterima jpg, jpeg, png, dan pdf dengan ukuran maksimal 2 MB.

```php
<?php

namespace Modules\Peminjaman\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;

class PeminjamanRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * The file is required when creating, and optional when updating (keep the old file).

     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        $peminjaman = $this->route('peminjaman');

        return [
            'nama_peminjam' => ['required', 'string', 'max:255'],
            'judul_buku' => ['required', 'string', 'max:255'],
            'tanggal_pinjam' => ['required', 'date'],
            'tanggal_kembali' => ['nullable', 'date', 'after_or_equal:tanggal_pinjam'],
            'status' => ['required', Rule::in(['Dipinjam', 'Dikembalikan'])],
            'berkas' => [$peminjaman ? 'nullable' : 'required', 'file',
'mimes:jpg,jpeg,png,pdf', 'max:2048'],
        ];
    }

    /**
     * Get custom messages for validator errors.
     *
     * @return array<string, string>
     */
    public function messages(): array
    {
        return [
            'tanggal_kembali.after_or_equal' => 'Tanggal kembali tidak boleh sebelum
tanggal pinjam.',
            'berkas.required' => 'Berkas wajib diunggah.',
            'berkas.mimes' => 'Berkas harus berformat jpg, jpeg, png, atau pdf.',
            'berkas.max' => 'Ukuran berkas maksimal 2 MB.',
        ];
    }
}
```

Kode 20. PeminjamanRequest dengan aturan berkas.

Semua route peminjaman berada dalam grup auth dan role:admin,user. Route edit berupa GET yang mengembalikan JSON untuk mengisi modal, bukan halaman form.

```php
<?php

use Illuminate\Support\Facades\Route;
use Modules\Peminjaman\Http\Controllers\PeminjamanController;

Route::middleware(['auth', 'role:admin,user'])->group(function () {
    Route::get('peminjaman', [PeminjamanController::class, 'index'])->name('peminjaman.index');
    Route::get('peminjaman/data', [PeminjamanController::class, 'getData'])->name('peminjaman.data');
    Route::post('peminjaman', [PeminjamanController::class, 'store'])->name('peminjaman.store');
    Route::get('peminjaman/{peminjaman}/edit', [PeminjamanController::class, 'edit'])->name('peminjaman.edit');
    Route::put('peminjaman/{peminjaman}', [PeminjamanController::class, 'update'])->name('peminjaman.update');
    Route::delete('peminjaman/{peminjaman}', [PeminjamanController::class, 'destroy'])->name('peminjaman.destroy');
});
```

Kode 21. Route modul Peminjaman.

Controller selalu mengembalikan JSON. Method index hanya menampilkan halaman, sedangkan data tabel diambil lewat getData(). Method store menyimpan berkas ke disk public pada folder berkas_peminjaman dan mencatat nama aslinya.

```php
<?php

namespace Modules\Peminjaman\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\JsonResponse;
use Illuminate\Support\Facades\Storage;
use Illuminate\View\View;
use Modules\Peminjaman\Http\Requests\PeminjamanRequest;
use Modules\Peminjaman\Models\Peminjaman;

class PeminjamanController extends Controller
{
    /**
     * Display the page. Data is loaded via AJAX.
     */
    public function index(): View
    {
        return view('peminjaman::index');
    }

    /**
     * Return all loans as JSON.
     */
    public function getData(): JsonResponse
    {
        return response()->json(Peminjaman::latest()->get());
    }

    /**
     * Store a newly created loan together with its uploaded file.
     */
    public function store(PeminjamanRequest $request): JsonResponse
    {
        $data = $request->safe()->except('berkas');
        $file = $request->file('berkas');
        $data['berkas'] = $file->store('berkas_peminjaman', 'public');
        $data['nama_berkas'] = $file->getClientOriginalName();

        Peminjaman::create($data);

        return response()->json(['success' => 'Data berhasil disimpan.']);
    }

    /**
     * Return a single loan for the edit form.
     */
    public function edit(Peminjaman $peminjaman): JsonResponse
    {
        return response()->json($peminjaman);
    }
}
```

Kode 22. PeminjamanController, method index, getData, store, dan edit.

Pada update, berkas lama baru dihapus jika ada berkas baru yang diunggah. Method destroy menghapus berkas dari disk lebih dulu, lalu menghapus barisnya.

```php
class PeminjamanController extends Controller
{
    /**
     * Update the specified loan. The old file is replaced only when a new one is uploaded.
     */
    public function update(PeminjamanRequest $request, Peminjaman $peminjaman): JsonResponse
    {
        $data = $request->safe()->except('berkas');

        if ($request->hasFile('berkas')) {
            if ($peminjaman->berkas) {
                Storage::disk('public')->delete($peminjaman->berkas);
            }

            $file = $request->file('berkas');
            $data['berkas'] = $file->store('berkas_peminjaman', 'public');
            $data['nama_berkas'] = $file->getClientOriginalName();
        }

        $peminjaman->update($data);

        return response()->json(['success' => 'Data berhasil diperbarui.']);
    }

    /**
     * Remove the specified loan and its uploaded file.
     */
    public function destroy(Peminjaman $peminjaman): JsonResponse
    {
        if ($peminjaman->berkas) {
            Storage::disk('public')->delete($peminjaman->berkas);
        }

        $peminjaman->delete();

        return response()->json(['success' => 'Data berhasil dihapus.']);
    }
}
```

Kode 23. PeminjamanController, method update dan destroy.

C.7 Tampilan dan JavaScript AJAX

View peminjaman terdiri dari satu tabel yang awalnya hanya berisi tulisan Memuat data, serta satu modal Bootstrap yang dipakai untuk tambah maupun edit.

```
<div class="table-responsive">
    <table class="table table-hover" id="peminjaman-table">
        <thead>
            <tr>
                <th width="5%">No</th>
                <th>Nama Peminjam</th>
                <th>Judul Buku</th>
                <th>Tanggal Pinjam</th>
                <th>Tanggal Kembali</th>
                <th>Status</th>
                <th>Berkas</th>
                <th class="text-end">Aksi</th>
            </tr>
        </thead>
        <tbody>
            <tr>
                <td colspan="8" class="text-center text-muted py-5">
                    <div class="spinner-border spinner-border-sm me-2" role="status">
</div>
                    Memuat data...
                </td>
            </tr>
        </tbody>
    </table>
</div>
```

Kode 24. Tabel pada view index modul Peminjaman.

Form di dalam modal memuat satu input file dan satu elemen invalid-feedback untuk setiap input. Elemen inilah yang diisi JavaScript ketika server membalas 422.

```blade
<form id="peminjamanForm">
    @csrf
    <input type="hidden" name="id" id="id">

    @foreach ([
        'nama_peminjam' => ['label' => 'Nama Peminjam', 'type' => 'text', 'placeholder' => 'Nama anggota peminjam'],
        'judul_buku' => ['label' => 'Judul Buku', 'type' => 'text', 'placeholder' => 'Judul buku yang dipinjam'],
        'tanggal_pinjam' => ['label' => 'Tanggal Pinjam', 'type' => 'date', 'placeholder' => ''],
        'tanggal_kembali' => ['label' => 'Tanggal Kembali', 'type' => 'date', 'placeholder' => ''],
    ] as $name => $field)
        <div class="mb-3">
            <label for="{{ $name }}" class="form-label">
                {{ $field['label'] }}
                @if ($name !== 'tanggal_kembali')<span class="text-danger">*</span>@endif
            </label>
            <input type="{{ $field['type'] }}" class="form-control" id="{{ $name }}" name="{{ $name }}" placeholder="{{ $field['placeholder'] }}" @required($name !== 'tanggal_kembali')>
            <div class="invalid-feedback" id="{{ $name }}-error"></div>
        </div>
    @endforeach

    <div class="mb-3">
        <label for="status" class="form-label">Status <span class="text-danger">*</span></label>
        <select class="form-select" id="status" name="status" required>
            <option value="Dipinjam">Dipinjam</option>
            <option value="Dikembalikan">Dikembalikan</option>
        </select>
        <div class="invalid-feedback" id="status-error"></div>
    </div>

    <div class="mb-3">
        <label for="berkas" class="form-label">
            Berkas Pendukung <span class="text-danger" id="berkas-required">*</span>
        </label>
        <div class="small mb-2 d-none" id="berkas-current">
            Berkas saat ini: <a href="#" target="_blank" rel="noopener" id="berkas-current-link"></a>
        </div>
        <input type="file" class="form-control" id="berkas" name="berkas" accept=".jpg,.jpeg,.png,.pdf">
        <div class="invalid-feedback" id="berkas-error"></div>
        <div class="form-text" id="berkas-help">Format jpg, jpeg, png, atau pdf. Maksimal 2 MB.</div>
    </div>
</form>
```

Kode 25. Form di dalam modal Peminjaman.

Bagian awal skrip menyimpan URL route, memasang header X-CSRF-TOKEN dan Accept application/json untuk semua request lewat $.ajaxSetup, dan menyediakan escapeHtml agar data yang tampil di tabel tidak dieksekusi sebagai HTML.

```
const urls = {
    data: "{{ route('peminjaman.data') }}",
    store: "{{ route('peminjaman.store') }}",
    edit: "{{ route('peminjaman.edit', ':id') }}",
    update: "{{ route('peminjaman.update', ':id') }}",
    destroy: "{{ route('peminjaman.destroy', ':id') }}",
};

let records = [];

// Set CSRF token untuk semua request AJAX
$.ajaxSetup({
    headers: {
        'X-CSRF-TOKEN': $('meta[name="csrf-token"]').attr('content'),
        'Accept': 'application/json'
    }
});

// Escape karakter HTML (termasuk tanda kutip untuk atribut) agar data user tidak
dieksekusi sebagai script (XSS)
function escapeHtml(value) {
    return String(value ?? '')
        .replace(/&/g, '&amp;')
        .replace(/</g, '&lt;')
        .replace(/>/g, '&gt;')
        .replace(/"/g, '&quot;')
        .replace(/'/g, '&#039;');
}
```

Kode 26. Pengaturan URL, header CSRF, dan escapeHtml.

Tautan berkas dibuat oleh fileLink() dan membuka berkas di tab baru. Fungsi setFileMode() mengatur input berkas, yaitu wajib saat tambah dan opsional saat edit, sambil menampilkan tautan berkas yang sedang tersimpan.

```
// Link untuk membuka berkas di tab baru
function fileLink(item) {
    if (! item.berkas_url) {
        return '<span class="text-muted">-</span>';
    }

    const isPdf = (item.nama_berkas || item.berkas).toLowerCase().endsWith('.pdf');
    const icon = isPdf ? 'bi-file-earmark-pdf text-danger' : 'bi-file-earmark-image text-primary';

    return `<a href="${escapeHtml(item.berkas_url)}" target="_blank" rel="noopener" class="small text-decoration-none">
        <i class="bi ${icon} me-1"></i>${escapeHtml(item.nama_berkas || 'Lihat berkas')}
    </a>`;
}

function statusBadge(status) {
    const badge = status === 'Dikembalikan' ? 'text-bg-success' : 'text-bg-warning';

    return `<span class="badge ${badge}">${escapeHtml(status)}</span>`;
}

// Mode input berkas: wajib saat tambah, opsional saat edit
function setFileMode(item = null) {
    $('#berkas').val('');
    $('#berkas-required').toggleClass('d-none', item !== null);
    $('#berkas-current').toggleClass('d-none', ! item?.berkas_url);
    $('#berkas-help').text(item
        ? 'Kosongkan jika tidak ingin mengganti berkas. Format jpg, jpeg, png, atau pdf. Maksimal 2 MB.'
        : 'Format jpg, jpeg, png, atau pdf. Maksimal 2 MB.');

    if (item?.berkas_url) {
        $('#berkas-current-link').attr('href', item.berkas_url).text(item.nama_berkas || 'Lihat berkas');
    }
}
```

Kode 27. Fungsi fileLink, statusBadge, dan setFileMode.

renderTable() menyaring data di sisi klien berdasarkan kata kunci pencarian lalu mengisi tabel. Datanya sendiri dimuat oleh loadData() lewat GET ke peminjaman/data.

```
// Tampilkan data ke tabel sesuai kata kunci pencarian
function renderTable() {
    const keyword = $('#search').val().trim().toLowerCase();
    const rows = records.filter((item) => [item.nama_peminjam, item.judul_buku,
item.tanggal_pinjam, item.tanggal_kembali, item.status]
        .some((value) => (value || '').toLowerCase().includes(keyword))
    );

    $('#total-count').text(records.length);

    if (records.length === 0) {
        $('#peminjaman-table tbody').html(emptyRow('bi-journal-x', 'Belum ada data
peminjaman. Klik <strong>Tambah Peminjaman</strong> untuk memulai.'));
        return;
    }

    if (rows.length === 0) {
        $('#peminjaman-table tbody').html(emptyRow('bi-search', 'Tidak ada data yang cocok
dengan pencarian.'));
        return;
    }

    const html = rows.map((item, index) => `
        <tr>
            <td class="text-muted">${index + 1}</td>
            <td class="fw-medium">${escapeHtml(item.nama_peminjam)}</td>
            <td>${escapeHtml(item.judul_buku)}</td>
            <td class="small">${escapeHtml(item.tanggal_pinjam || '-')}</td>
            <td class="small">${escapeHtml(item.tanggal_kembali || '-')}</td>
            <td>${statusBadge(item.status)}</td>
            <td>${fileLink(item)}</td>
            <td class="text-end text-nowrap">
                <button data-id="${item.id}" class="edit btn btn-sm btn-outline-primary">
                    <i class="bi bi-pencil"></i> Edit
                </button>
                <button data-id="${item.id}" data-nama="${escapeHtml(item.nama_peminjam)}"
class="delete btn btn-sm btn-outline-danger">
                    <i class="bi bi-trash"></i> Hapus
                </button>
            </td>
        </tr>
    `).join('');

    $('#peminjaman-table tbody').html(html);
}
```

Kode 28. Fungsi renderTable.

```
// Muat data dari server
function loadData() {
    $.ajax({
        url: urls.data,
        type: 'GET',
        dataType: 'json',
        success: function (data) {
            records = data;
            renderTable();
        },
        error: function (xhr) {
            console.error('Error:', xhr.responseText);
            $('#peminjaman-table tbody').html(emptyRow('bi-wifi-off', 'Gagal memuat data. Silakan refresh halaman.'));
        }
    });
}

loadData();
$('#search').on('input', renderTable);
```

Kode 29. Fungsi loadData dan pencarian.

Tombol Simpan membuat FormData dari form supaya berkas ikut terkirim. Request dikirim dengan processData false dan contentType false agar jQuery tidak mengubah FormData menjadi query string. Saat mengedit, field _method bernilai PUT ditambahkan. Jika server membalas 422, pesan pertama untuk tiap field dipasang di bawah inputnya.

```
// Simpan data (create / update)
$('#saveBtn').click(function () {
    const btn = $(this);
    const id = $('#id').val();

    btn.prop('disabled', true);
    btn.find('.submit-text').text('Menyimpan...');
    btn.find('.spinner-border').removeClass('d-none');
    clearErrors();

    // FormData dipakai (bukan serialize) agar file ikut terkirim
    const formData = new FormData(document.getElementById('peminjamanForm'));

    if (id) {
        formData.append('_method', 'PUT');
    }

    $.ajax({
        url: id ? urls.update.replace(':id', id) : urls.store,
        type: 'POST',
        data: formData,
        processData: false,
        contentType: false,
        dataType: 'json',
        success: function (response) {
            peminjamanModal.hide();
            loadData();
            showAlert(response.success);
        },
        error: function (xhr) {
            if (xhr.status === 422) {
                $.each(xhr.responseJSON.errors, function (key, value) {
                    $(`#${key}`).addClass('is-invalid');
                    $(`#${key}-error`).text(value[0]);
                });
                $('#peminjamanForm .is-invalid:first').focus();
            } else {
                console.error('Error:', xhr.responseText);
                peminjamanModal.hide();
                showAlert('Terjadi kesalahan. Silakan coba lagi.', 'danger');
            }
        },
        complete: function () {
            btn.prop('disabled', false);
            btn.find('.submit-text').text('Simpan');
            btn.find('.spinner-border').addClass('d-none');
        }
    });
});
```

Kode 30. Penyimpanan data dengan FormData dan penanganan error 422.

Tombol Edit mengambil data lewat GET lalu mengisi modal. Tombol Hapus membuka modal konfirmasi, dan request DELETE baru dikirim setelah pengguna menekan Ya, hapus.

```
// Edit data
$(document).on('click', '.edit', function () {
    const id = $(this).data('id');

    $.get(urls.edit.replace(':id', id), function (data) {
        $('#peminjamanModalLabel').text('Edit Peminjaman');
        $('#id').val(data.id);
        $('#nama_peminjam').val(data.nama_peminjam);
        $('#judul_buku').val(data.judul_buku);
        $('#tanggal_pinjam').val(data.tanggal_pinjam);
        $('#tanggal_kembali').val(data.tanggal_kembali);
        $('#status').val(data.status);
        setFileMode(data);
        peminjamanModal.show();
    }).fail(function () {
        showAlert('Gagal memuat data untuk diedit.', 'danger');
    });
});

// Hapus data
$(document).on('click', '.delete', function () {
    const id = $(this).data('id');
    const nama = $(this).data('nama');

    confirmDelete(`Data peminjaman "${nama}" akan dihapus dan tidak bisa
dikembalikan.`).then(function (confirmed) {
        if (! confirmed) {
            return;
        }

        $.ajax({
            type: 'DELETE',
            url: urls.destroy.replace(':id', id),
            dataType: 'json',
            success: function (response) {
                loadData();
                showAlert(response.success);
            },
            error: function () {
                showAlert('Gagal menghapus data. Silakan coba lagi.', 'danger');
            }
        });
    });
});
```

Kode 31. Penanganan tombol Edit dan Hapus.

C.8 Modul UserManagement (CRUD Synchronous)

Route modul ini memakai Route::resource tanpa show, dengan prefix admin dan awalan nama admin. Seluruhnya berada dalam grup role:admin, sehingga halaman kelola user hanya bisa dibuka admin.

```php
<?php

use Illuminate\Support\Facades\Route;
use Modules\UserManagement\Http\Controllers\UserManagementController;

Route::middleware(['auth', 'role:admin'])
    ->prefix('admin')
    ->name('admin.')
    ->group(function () {
        Route::resource('users', UserManagementController::class)->except('show');
    });
```

Kode 32. Route modul UserManagement.

StoreUserRequest mewajibkan nama, email unik, role, dan password dengan konfirmasi. Pada UpdateUserRequest password boleh kosong. Aturan yang saya anggap paling penting ada pada role: ketika admin sedang mengubah akunnya sendiri, Rule::when membatasi pilihan hanya ke admin, jadi permintaan menurunkan role sendiri ditolak.

```php
<?php

namespace Modules\UserManagement\Http\Requests;

use App\Enums\UserRole;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Password;

class UpdateUserRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * An admin cannot demote their own account, so they never lock themselves out.
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        $user = $this->route('user');
        $isEditingOwnAccount = $user?->is($this->user()) ?? false;

        return [
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'email', 'max:255', Rule::unique('users', 'email')->ignore($user?->id)],
            'role' => [
                'required',
                Rule::enum(UserRole::class),
                Rule::when($isEditingOwnAccount, Rule::in([UserRole::Admin->value])),
            ],
            'password' => ['nullable', 'confirmed', Password::defaults()],
        ];
    }

    /**
     * Get custom messages for validator errors.
     *
     * @return array<string, string>
     */
    public function messages(): array
    {
        return [
            'role.in' => 'Anda tidak dapat menurunkan role akun Anda sendiri.',
        ];
    }
}
```

Kode 33. UpdateUserRequest dengan batasan role akun sendiri.

Method index mendukung pencarian nama atau email serta filter role.

```php
<?php

namespace Modules\UserManagement\Http\Controllers;

use App\Enums\UserRole;
use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\View\View;
use Modules\UserManagement\Http\Requests\StoreUserRequest;
use Modules\UserManagement\Http\Requests\UpdateUserRequest;

class UserManagementController extends Controller
{
    /**
     * Display a listing of users, optionally filtered by keyword and role.
     */
    public function index(Request $request): View
    {
        $search = $request->string('search')->trim()->toString();
        $role = UserRole::tryFrom($request->string('role')->toString());

        $users = User::query()
            ->when($search !== '', function (Builder $query) use ($search) {
                $query->where(function (Builder $query) use ($search) {
                    $query->where('name', 'like', "%{$search}%")
                        ->orWhere('email', 'like', "%{$search}%");
                });
            })
            ->when($role, fn (Builder $query) => $query->where('role', $role))
            ->orderBy('name')
            ->paginate(10)
            ->withQueryString();

        return view('usermanagement::index', [
            'users' => $users,
            'search' => $search,
            'selectedRole' => $role,
            'roles' => UserRole::cases(),
        ]);
    }
}
```

Kode 34. UserManagementController, method index.

```php
class UserManagementController extends Controller
{
    /**
     * Show the form for creating a new user.
     */
    public function create(): View
    {
        return view('usermanagement::create', ['roles' => UserRole::cases()]);
    }

    /**
     * Store a newly created user.
     */
    public function store(StoreUserRequest $request): RedirectResponse
    {
        User::create($request->validated());

        return redirect()->route('admin.users.index')->with('success', 'User berhasil ditambahkan.');
    }

    /**
     * Show the form for editing the specified user.
     */
    public function edit(User $user): View
    {
        return view('usermanagement::edit', [
            'user' => $user,
            'roles' => UserRole::cases(),
        ]);
    }
}
```

Kode 35. UserManagementController, method create, store, dan edit.

Pada update, password hanya diubah jika kolomnya diisi. Method destroy memeriksa apakah user yang akan dihapus adalah akun yang sedang login. Jika ya, penghapusan ditolak dengan pesan error. Tombol Hapus untuk akun sendiri juga tidak ditampilkan di tabel, tetapi pemeriksaan di controller tetap dipertahankan.

```php
class UserManagementController extends Controller
{
    /**
     * Update the specified user. The password is only changed when filled in.
     */
    public function update(UpdateUserRequest $request, User $user): RedirectResponse
    {
        $attributes = $request->validated();

        if (blank($attributes['password'] ?? null)) {
            unset($attributes['password']);
        }

        $user->update($attributes);

        return redirect()->route('admin.users.index')->with('success', 'User berhasil diperbarui.');
    }

    /**
     * Remove the specified user. An admin cannot delete their own account.
     */
    public function destroy(Request $request, User $user): RedirectResponse
    {
        if ($user->is($request->user())) {
            return redirect()->route('admin.users.index')->with('error', 'Anda tidak dapat menghapus akun Anda sendiri.');
        }

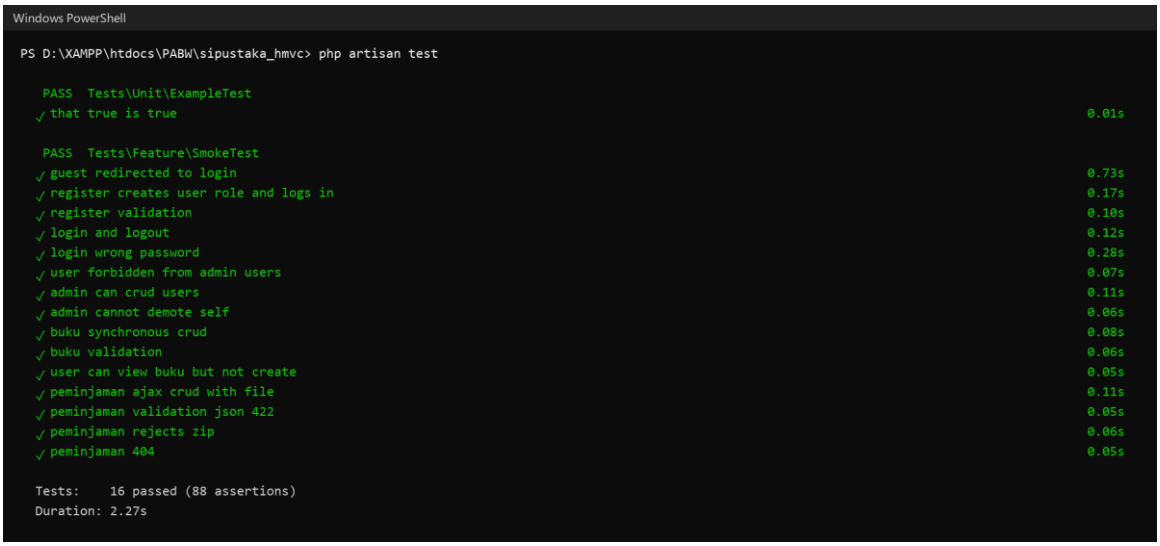
        $user->delete();

        return redirect()->route('admin.users.index')->with('success', 'User berhasil dihapus.');
    }
}
```

Kode 36. UserManagementController, method update dan destroy.

C.9 Pengujian Otomatis

Saya menulis feature test pada tests/Feature/SmokeTest.php. Test mencakup pengalihan tamu ke login, register yang selalu menghasilkan role user, 403 untuk user, admin yang tidak bisa menurunkan dirinya sendiri, CRUD buku, serta CRUD peminjaman dengan upload, penggantian, dan penghapusan berkas. phpunit.xml memakai SQLite in-memory sehingga data di MariaDB tidak tersentuh. Hasilnya 16 test lulus dengan 88 assertion.



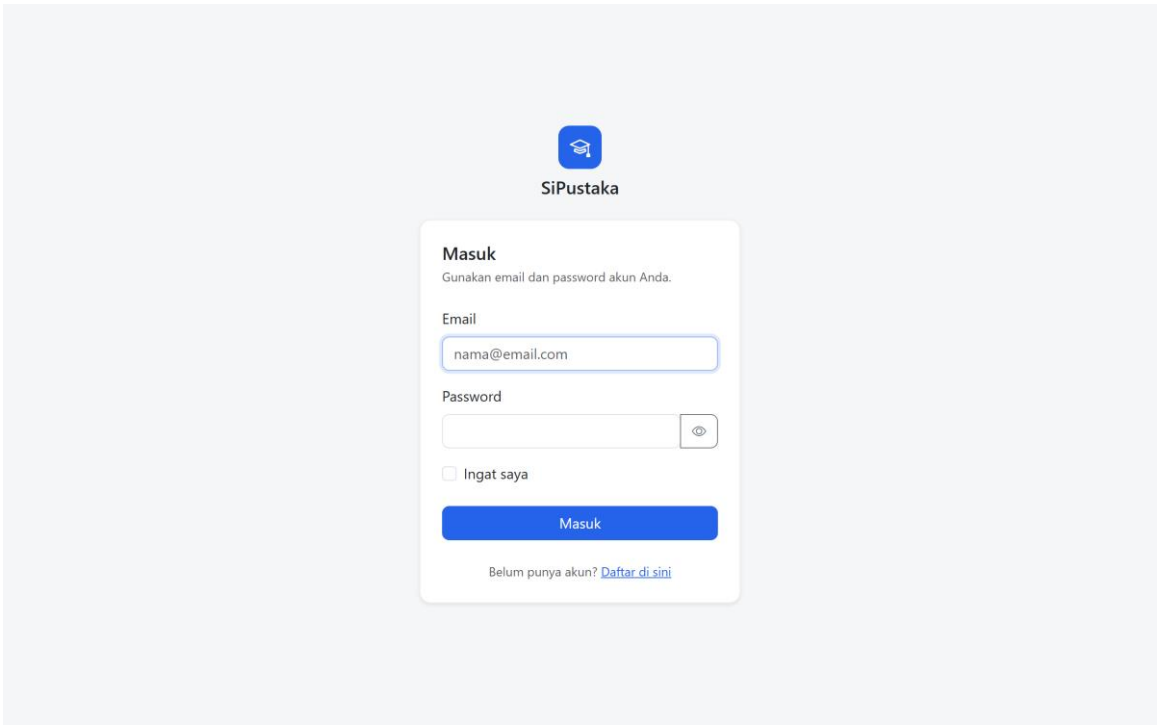
Gambar 2. Hasil php artisan test.

D. Hasil dan Dokumentasi Program

Tangkapan layar berikut diambil dari aplikasi yang berjalan di http://127.0.0.1:8150. Port 8000 sedang dipakai proses lain, jadi server saya jalankan dengan php artisan serve --port=8150. Data yang tampil adalah data uji yang saya masukkan sendiri.

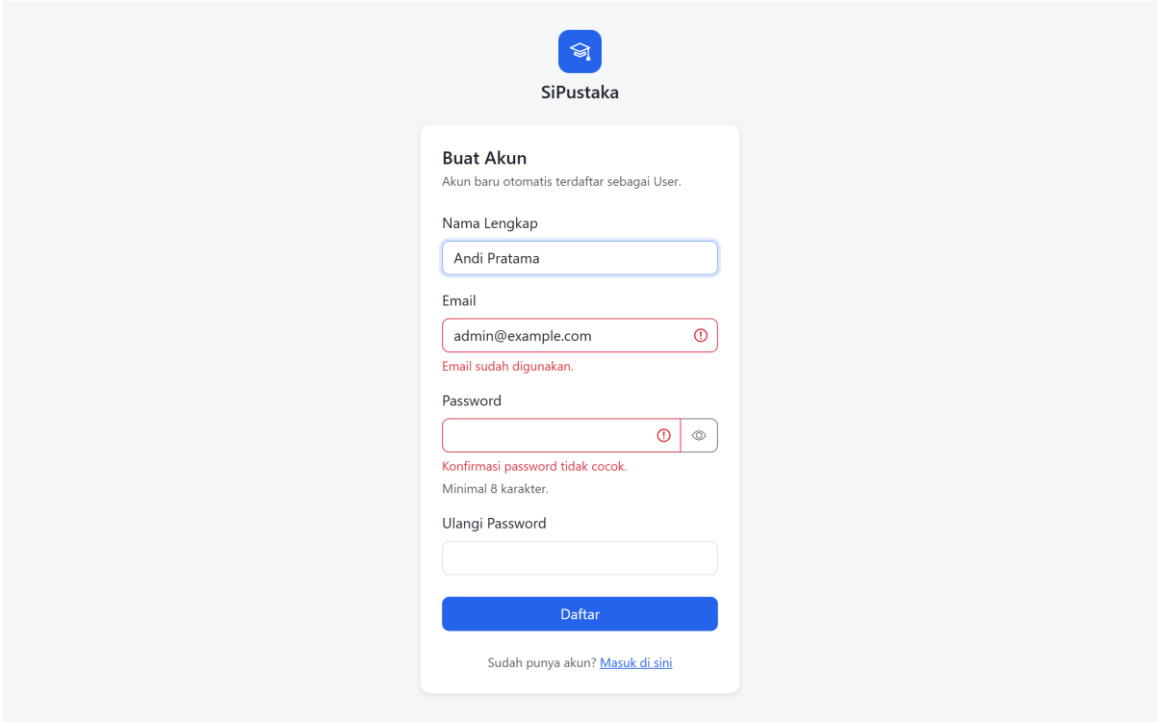
D.1 Login dan Register

Membuka halaman peminjaman tanpa login mengalihkan pengunjung ke halaman login.

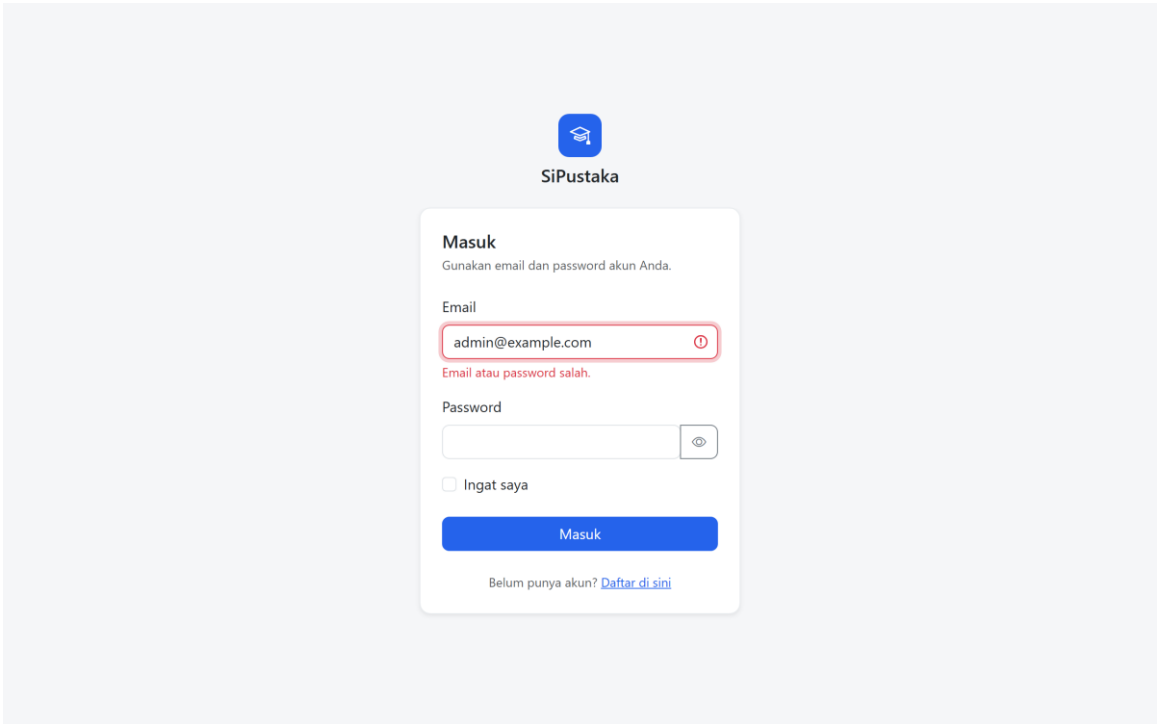


Gambar 3. Halaman login setelah /peminjaman dibuka tanpa login.

Form register menampilkan error di bawah input yang bermasalah. Pada contoh ini email sudah dipakai akun admin dan konfirmasi password tidak cocok.



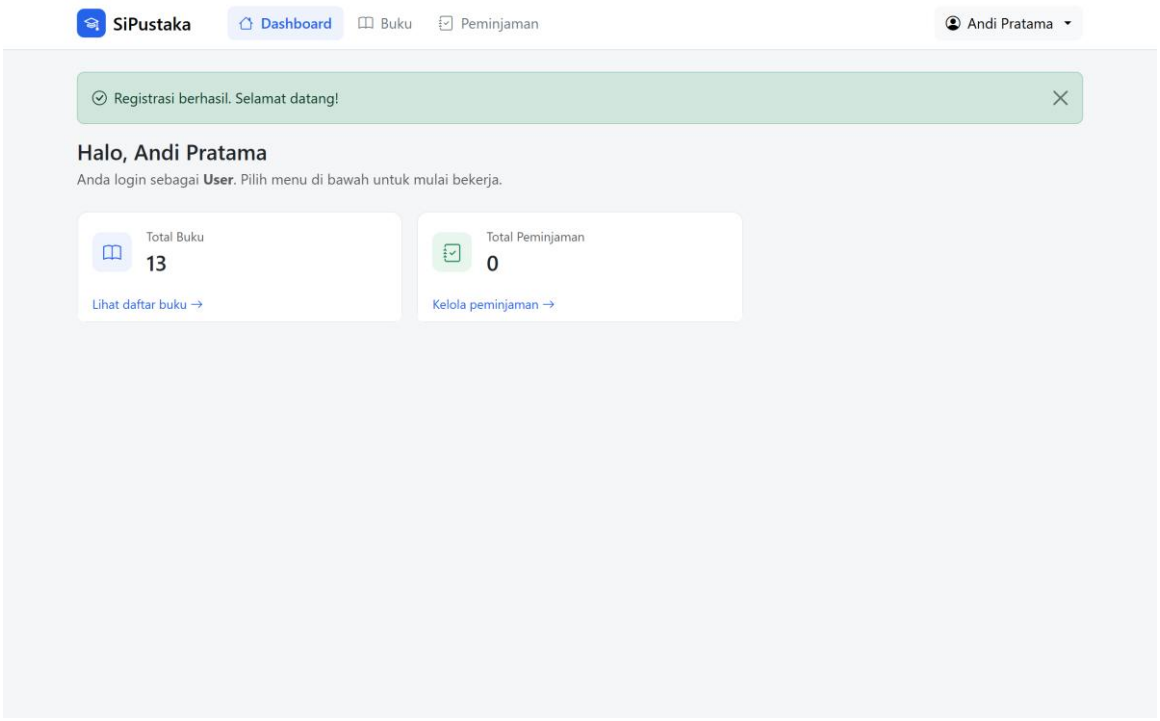
Gambar 4. Error validasi pada form register.



Gambar 5. Pesan error saat password login salah.

D.2 Dashboard dan Error 403

Setelah register berhasil, pengguna langsung masuk ke dashboard dengan pesan sukses. Dashboard user hanya memiliki dua kartu dan menu Kelola User tidak muncul.



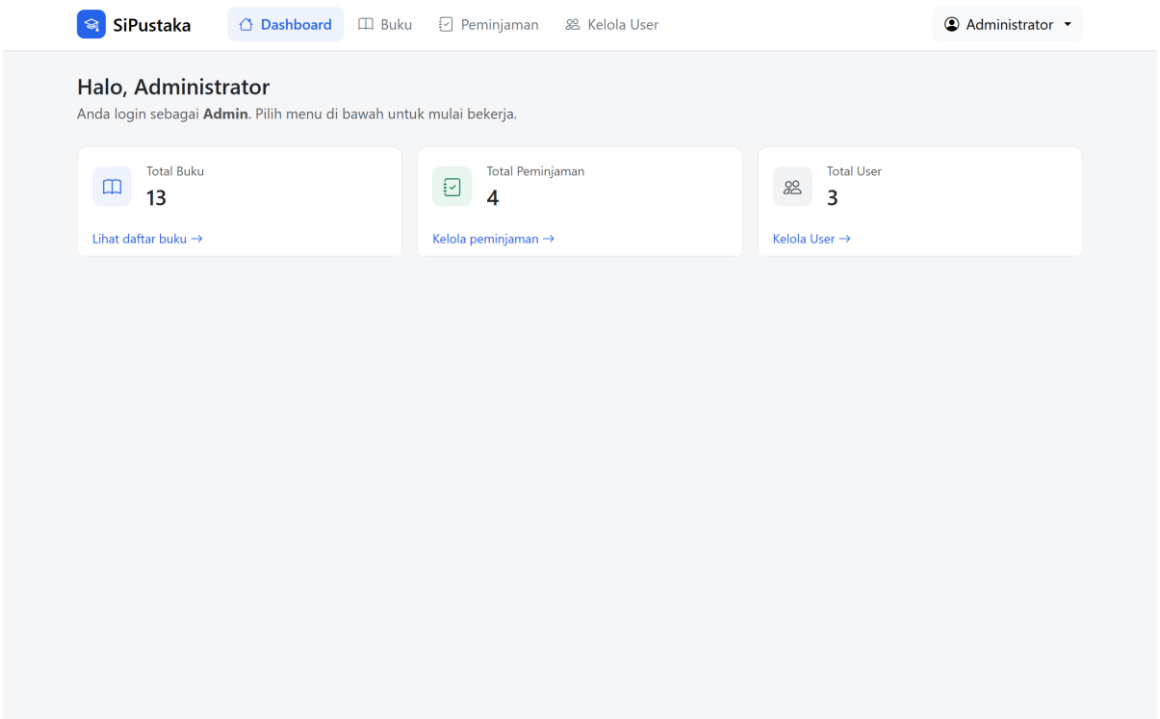
Gambar 6. Dashboard akun user setelah register.

Akun user yang membuka alamat /admin/users ditolak middleware dengan error 403.



Gambar 7. Error 403 saat user membuka halaman khusus admin.

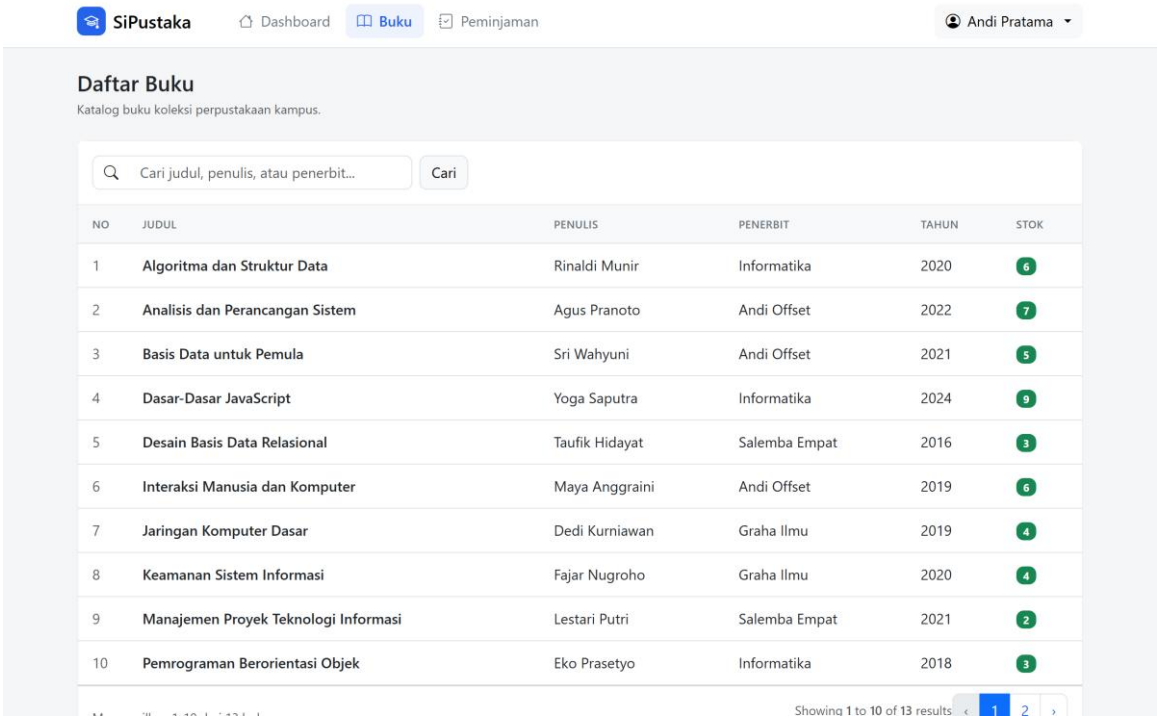
Dashboard admin memiliki satu kartu tambahan, yaitu total user, serta menu Kelola User pada navigasi.



Gambar 8. Dashboard akun admin.

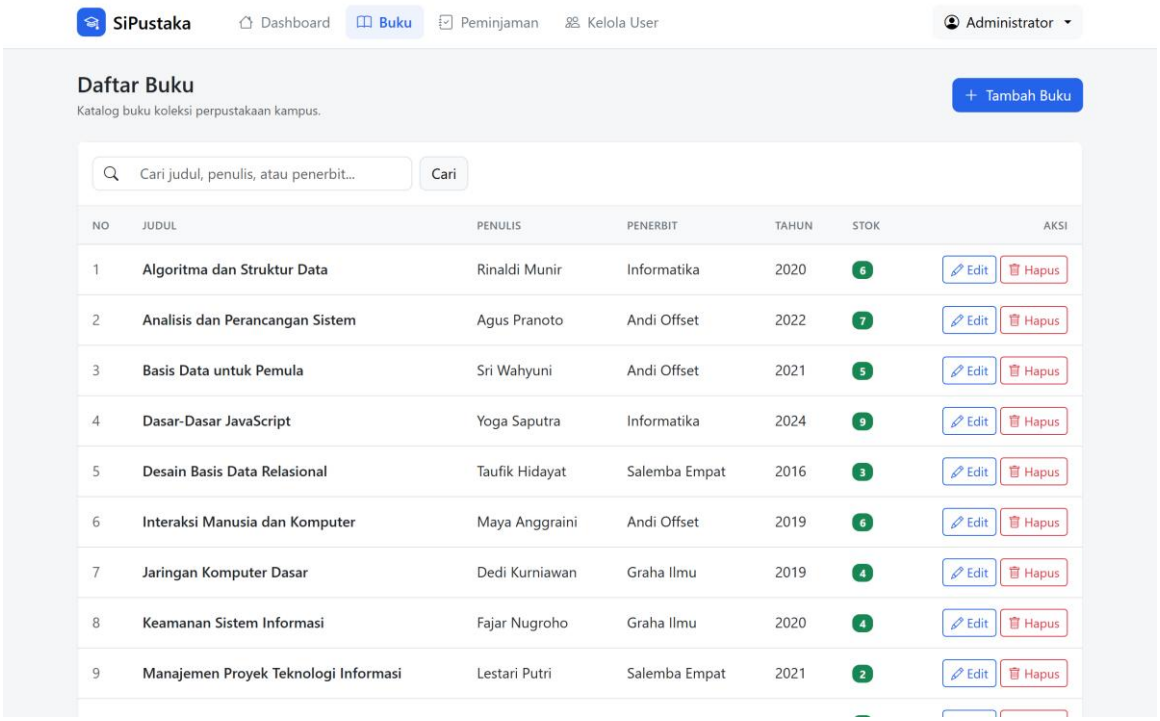
D.3 Modul Buku

User dapat melihat dan mencari buku, tetapi tidak ada tombol tambah, edit, maupun hapus pada halamannya.

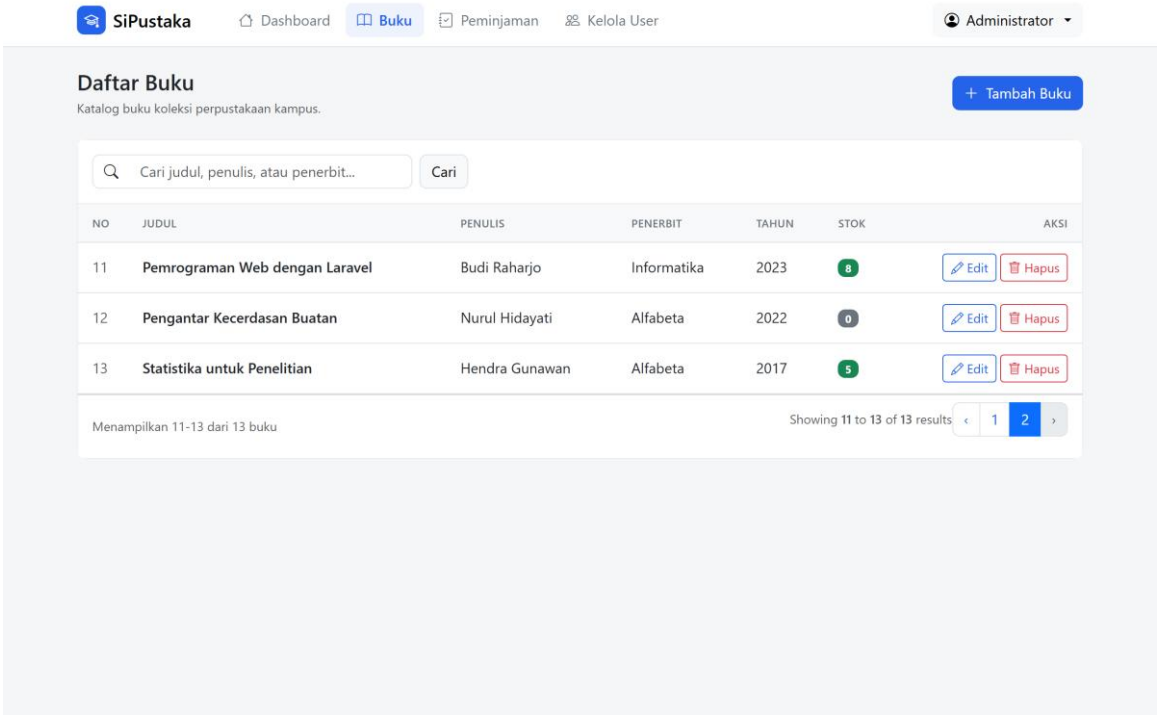


Gambar 9. Daftar buku pada akun user tanpa kolom Aksi.

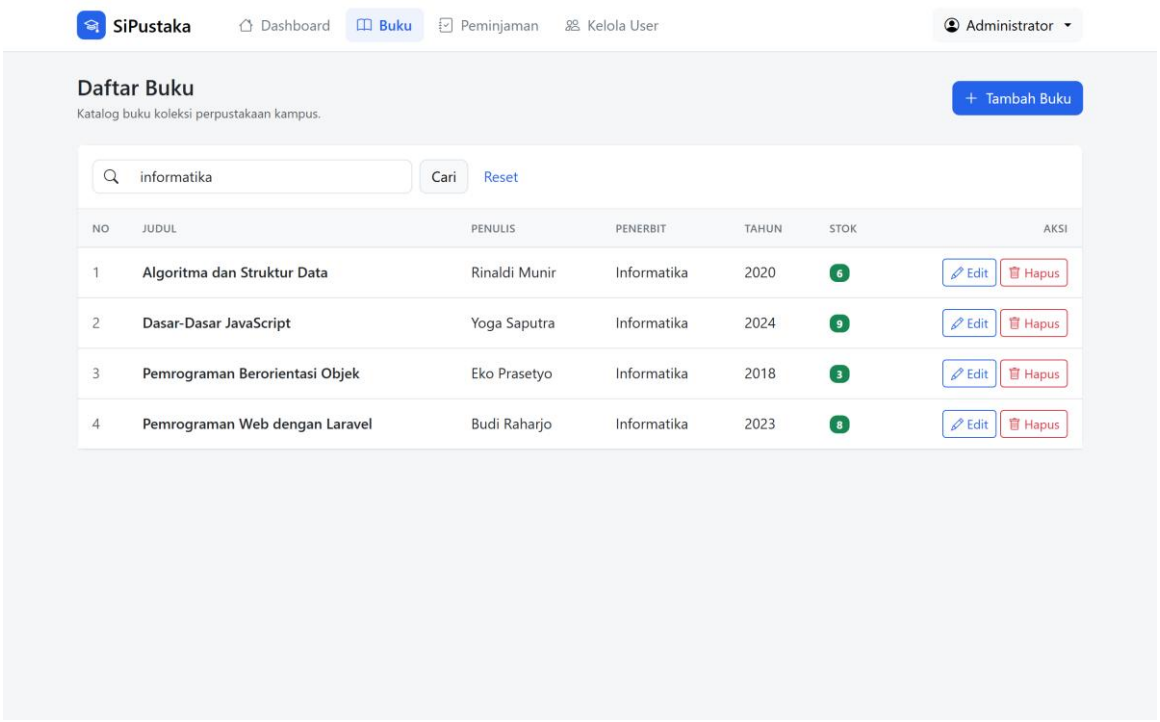
Admin melihat tombol Tambah Buku dan kolom Aksi. Data dibagi sepuluh per halaman dan halaman kedua memuat sisa tiga buku.



Gambar 10. Daftar buku pada akun admin.

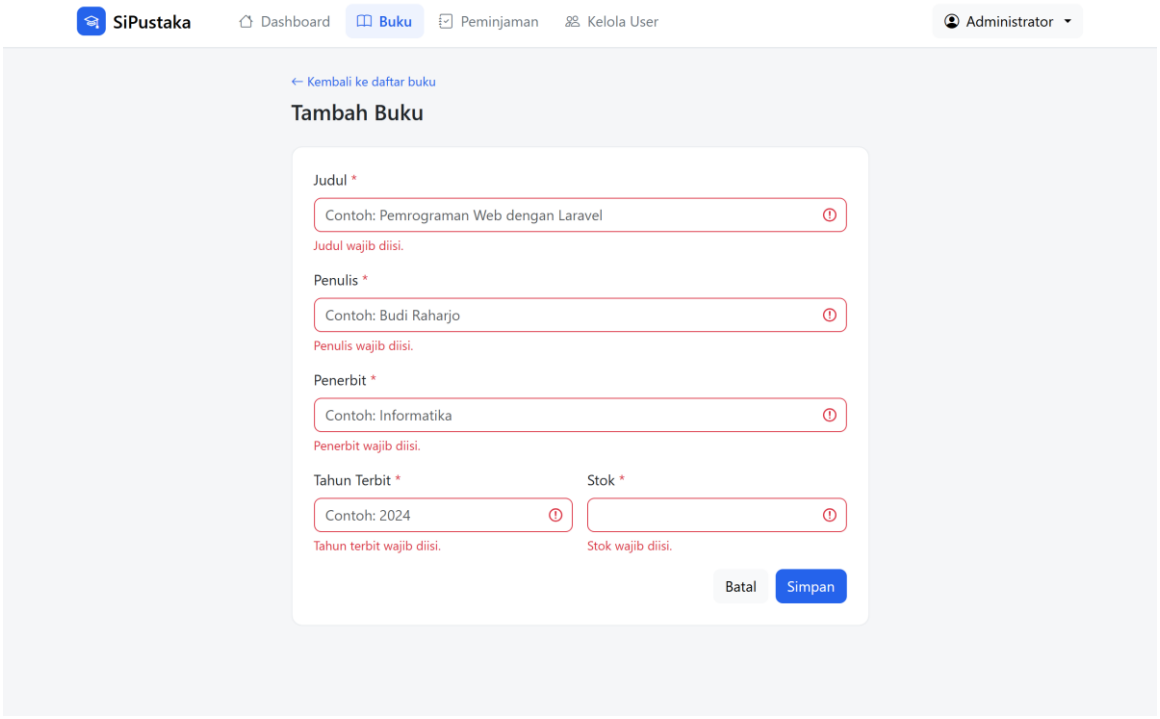


Gambar 11. Halaman kedua daftar buku.

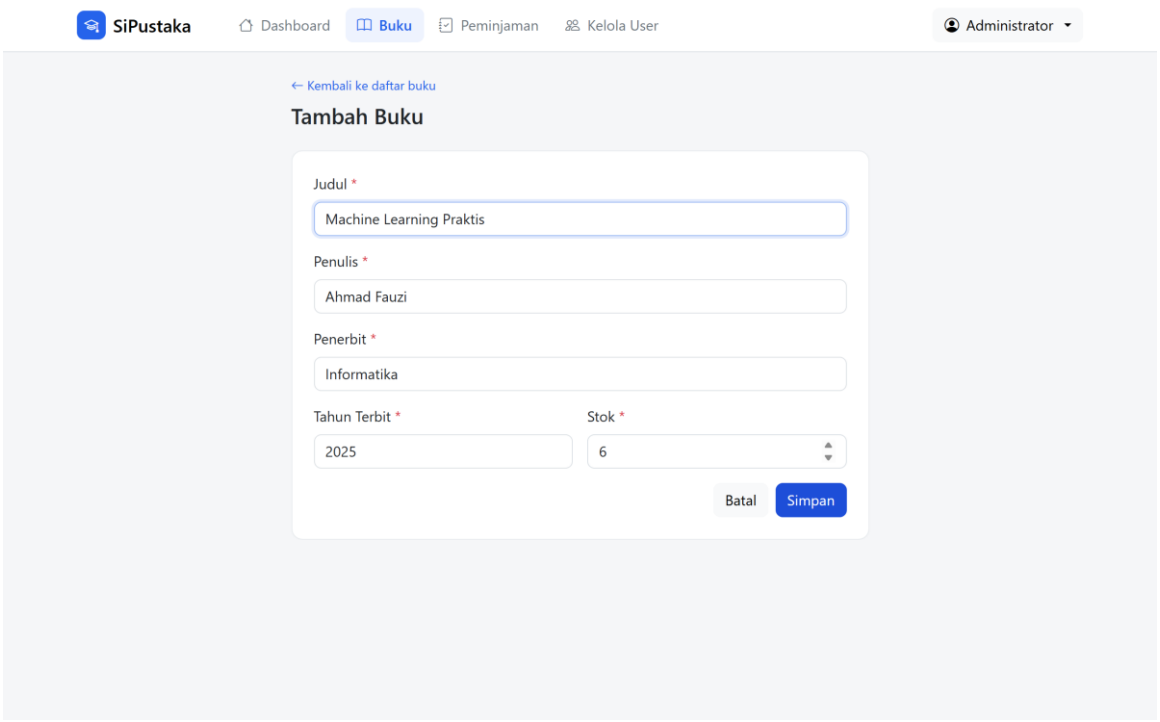


Gambar 12. Pencarian buku dengan kata kunci informatika.

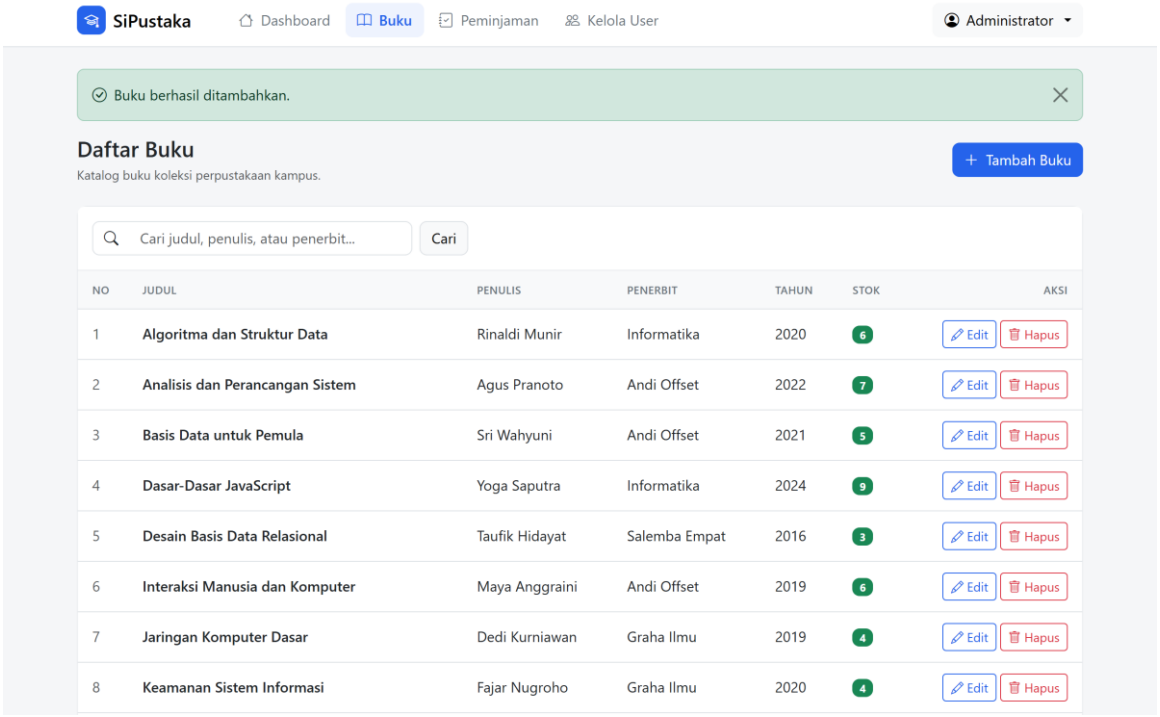
Form tambah yang dikirim kosong menampilkan pesan Form Request di bawah setiap input.



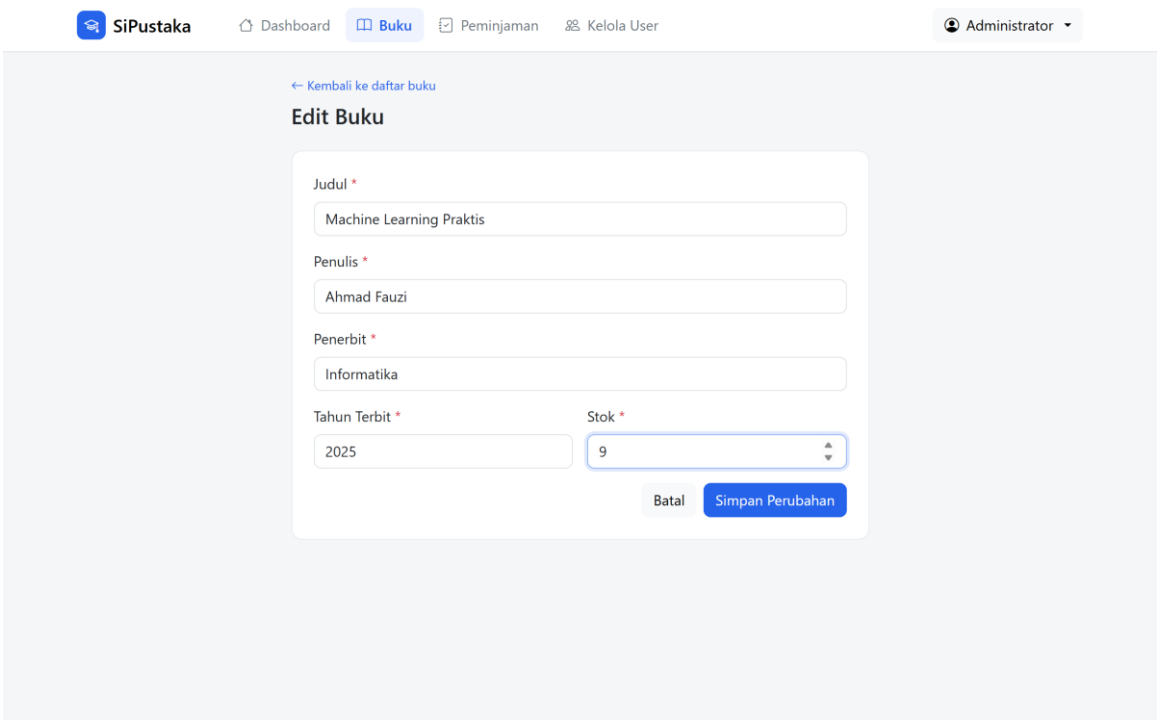
Gambar 13. Error validasi pada form tambah buku.



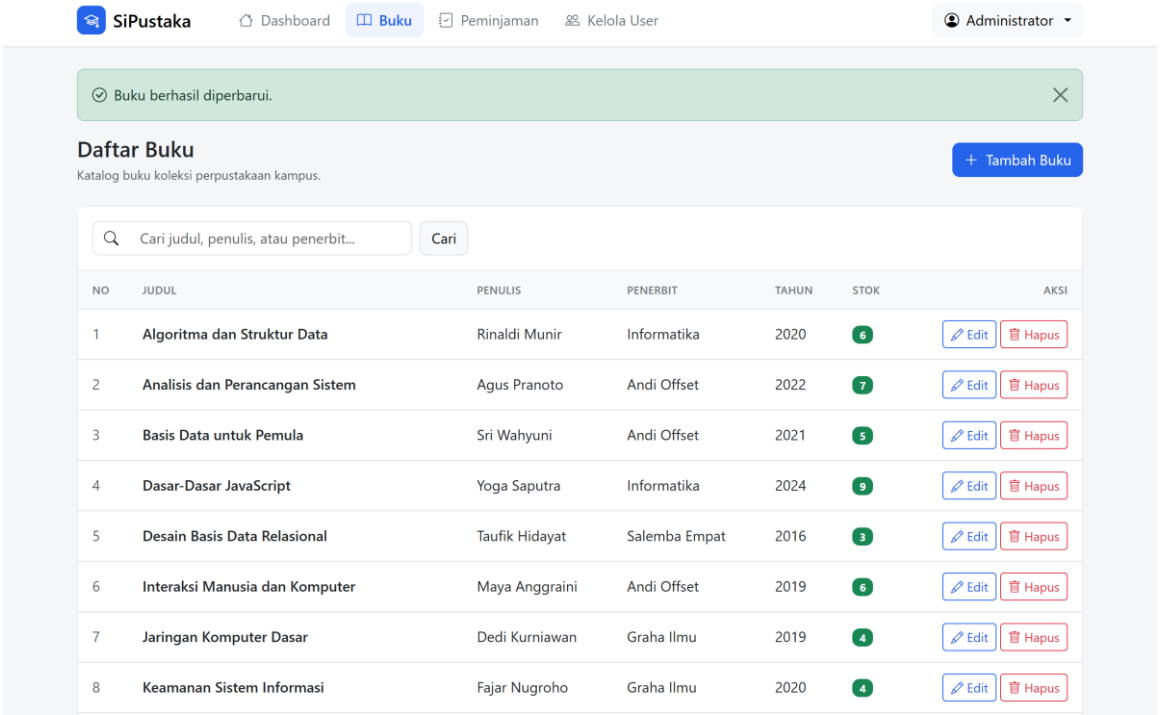
Gambar 14. Form tambah buku yang sudah diisi.



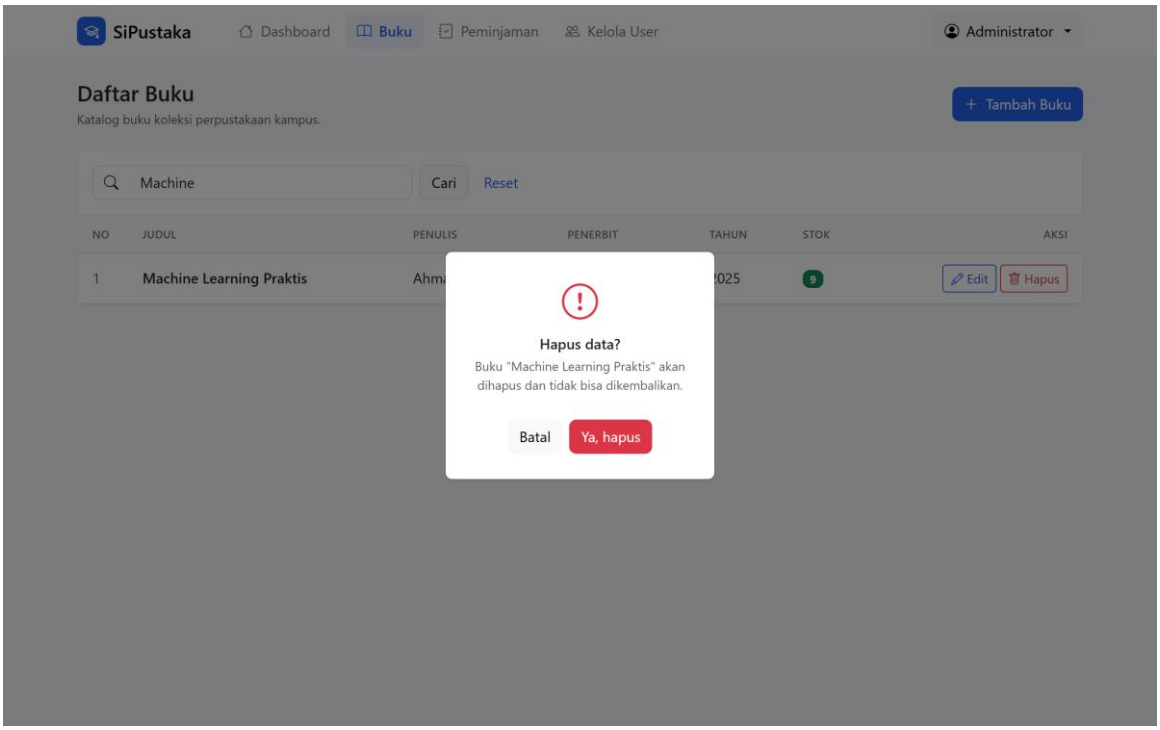
Gambar 15. Pesan berhasil setelah buku ditambahkan.



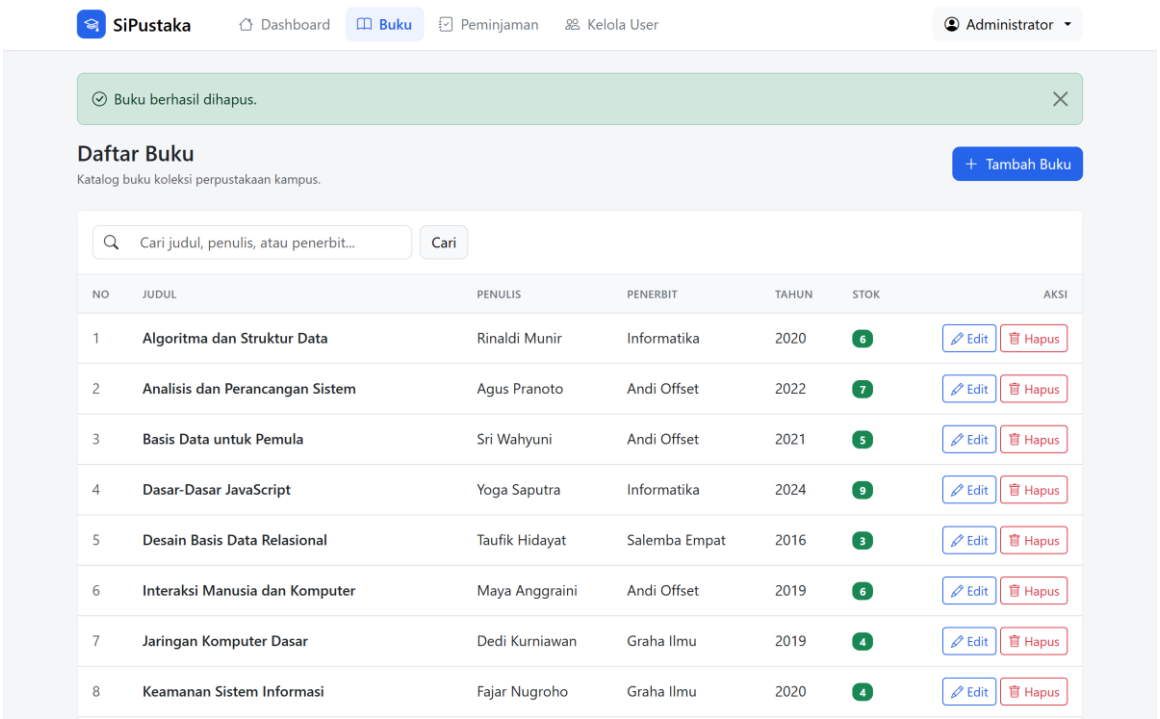
Gambar 16. Form edit buku dengan stok diubah menjadi 9.



Gambar 17. Pesan berhasil setelah buku diperbarui.



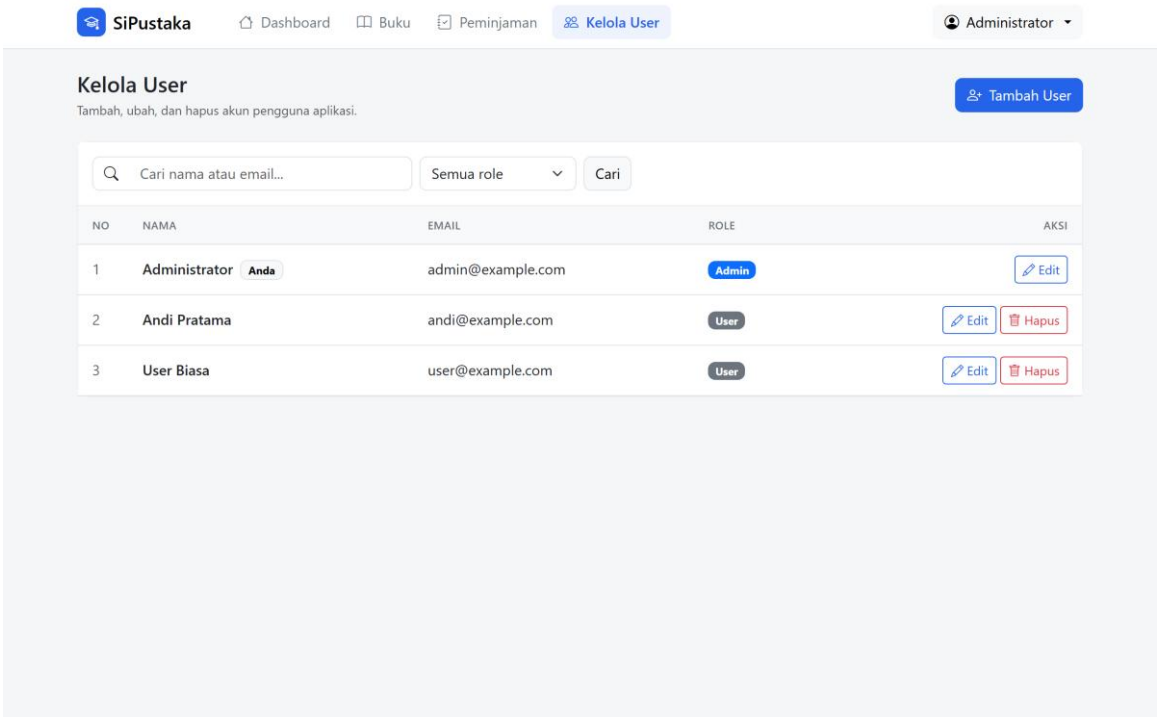
Gambar 18. Modal konfirmasi sebelum buku dihapus.



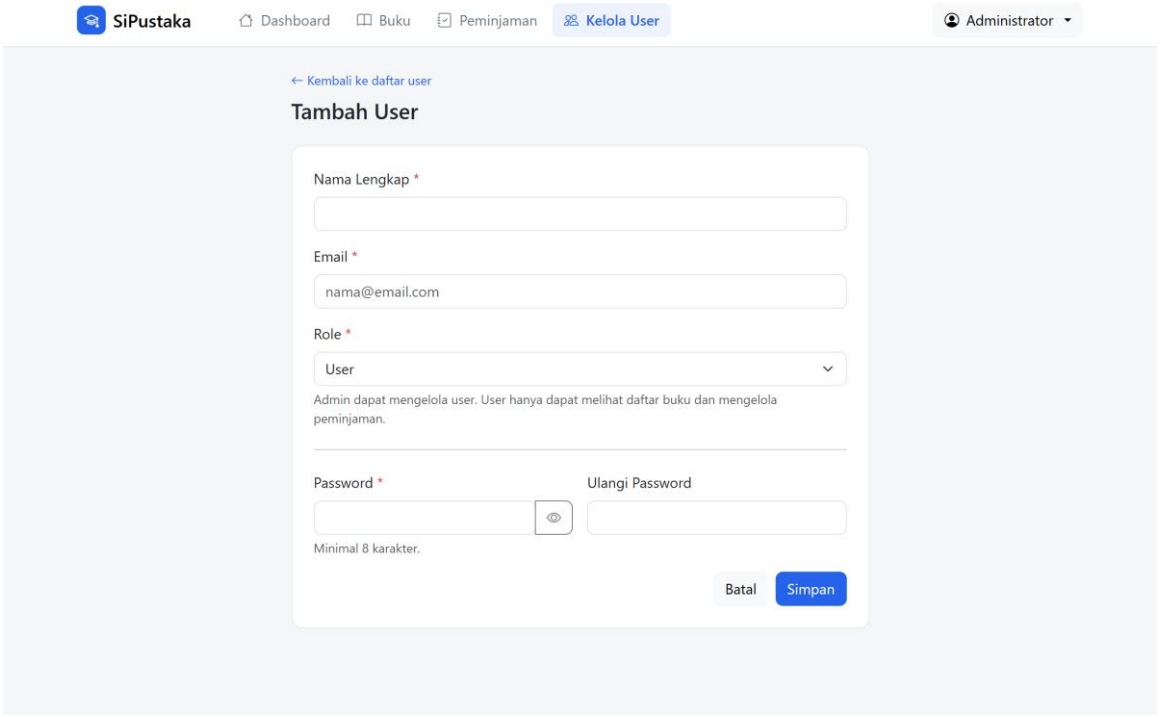
Gambar 19. Pesan berhasil setelah buku dihapus.

D.4 Modul UserManagement

Akun yang sedang login diberi badge Anda dan tidak punya tombol Hapus.

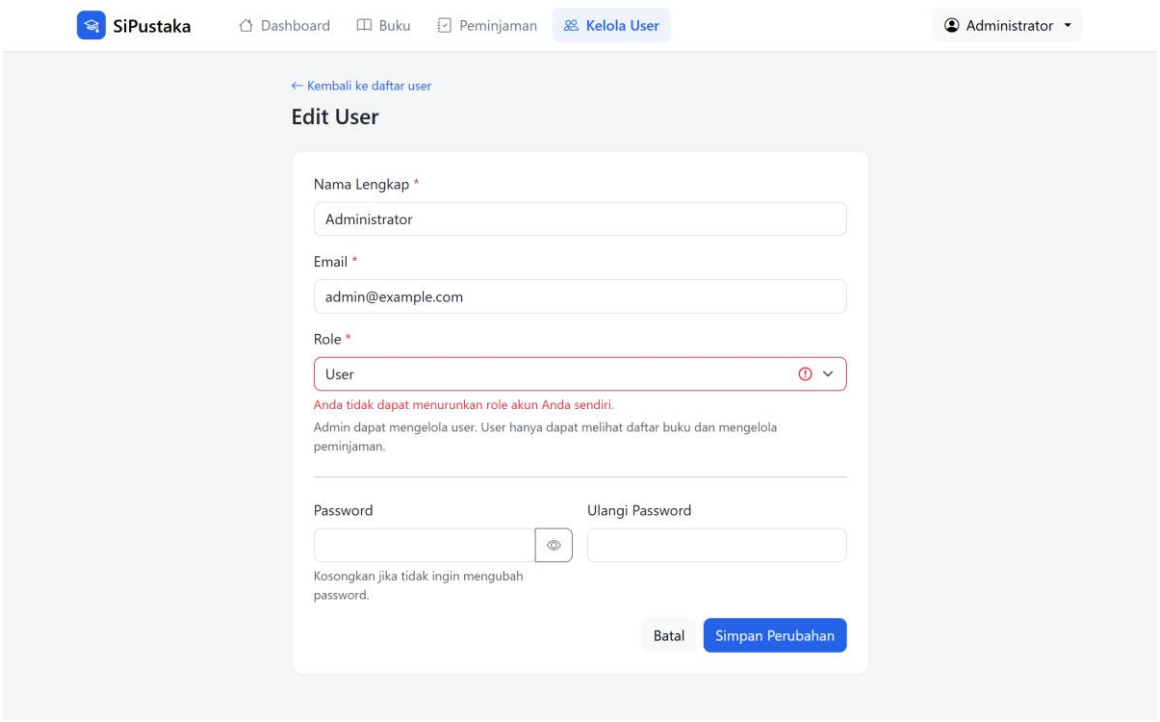


Gambar 20. Daftar user pada halaman Kelola User.



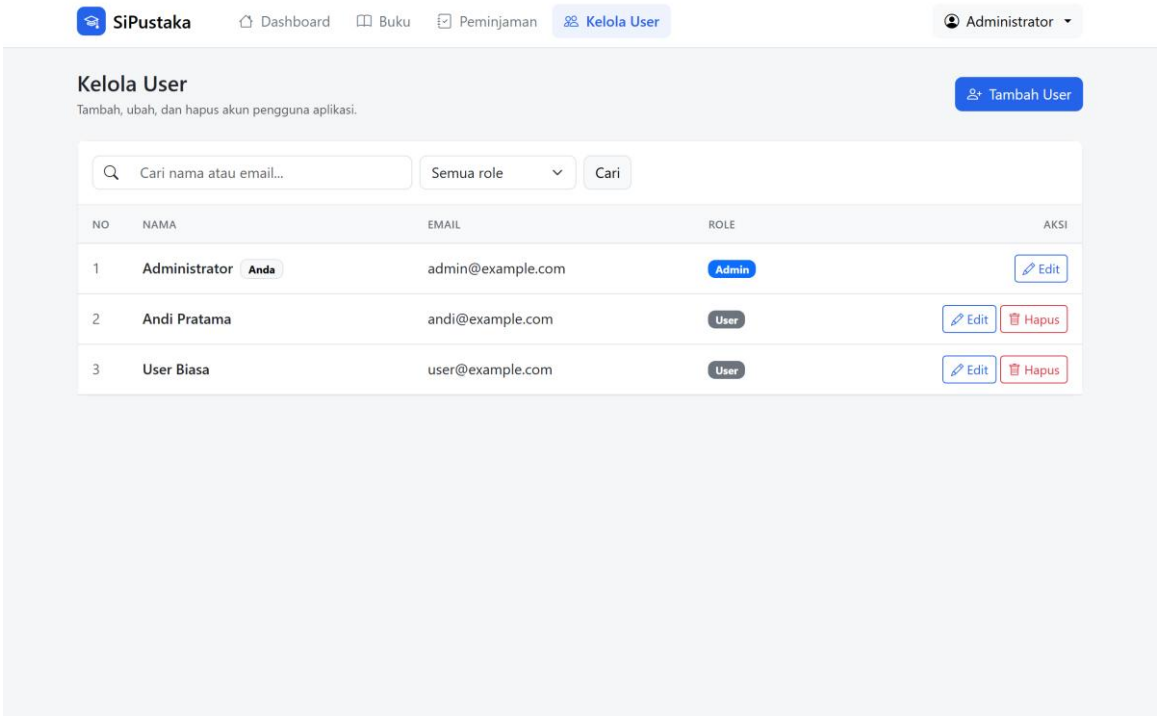
Gambar 21. Form tambah user.

Saat admin mengedit akunnya sendiri lalu memilih role User, server menolak dan menampilkan pesan dari UpdateUserRequest di bawah input role.

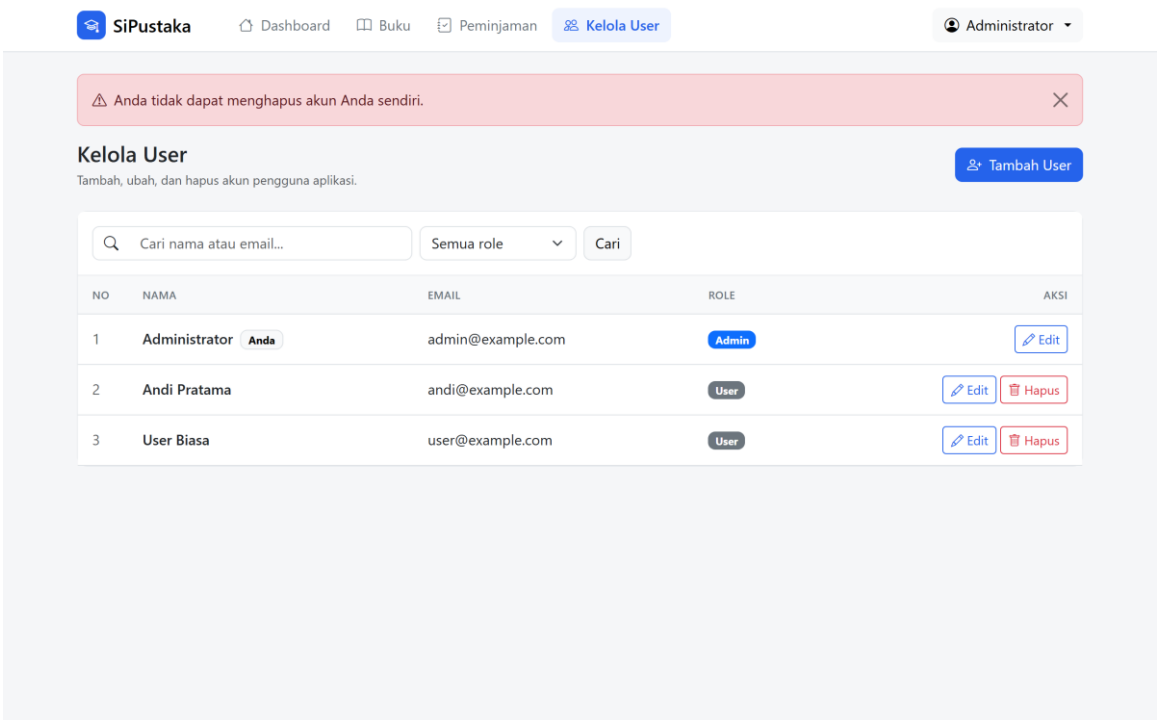


Gambar 22. Error saat admin mencoba menurunkan role akunnya sendiri.

Karena tombol Hapus untuk akun sendiri memang tidak ditampilkan, penolakan di sisi server saya buktikan dengan mengirim request DELETE ke akun sendiri lewat form yang saya sisipkan dengan JavaScript. Server menolak dan akun admin tetap ada.



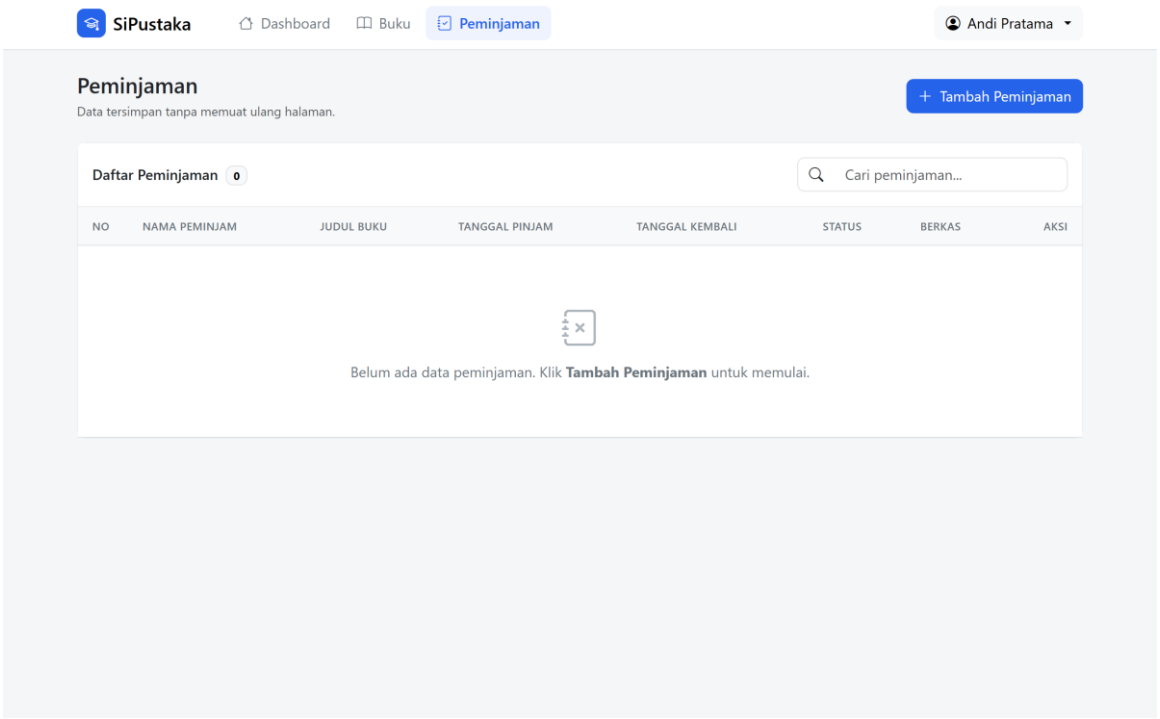
Gambar 23. Tabel user tanpa tombol Hapus pada akun sendiri.



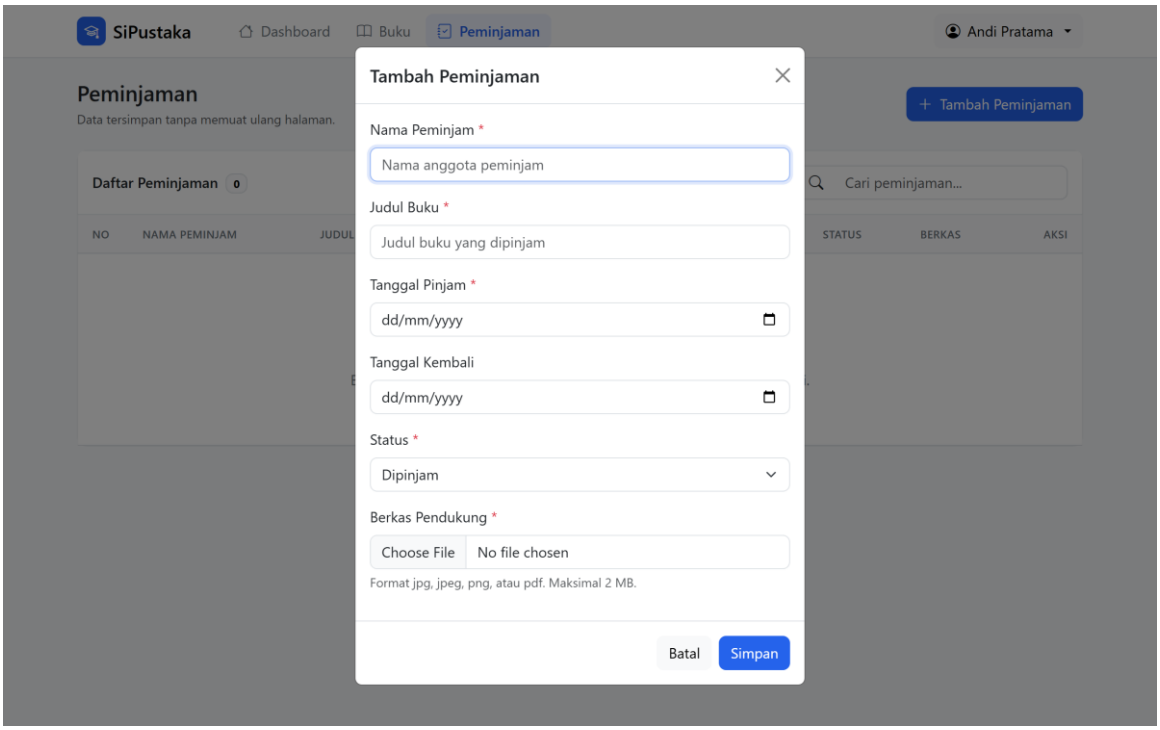
Gambar 24. Penghapusan akun sendiri ditolak oleh server.

D.5 Modul Peminjaman dengan AJAX

Halaman peminjaman dimuat sekali, lalu data tabel diambil lewat AJAX. Pada kondisi belum ada data, tabel menampilkan keterangan kosong.

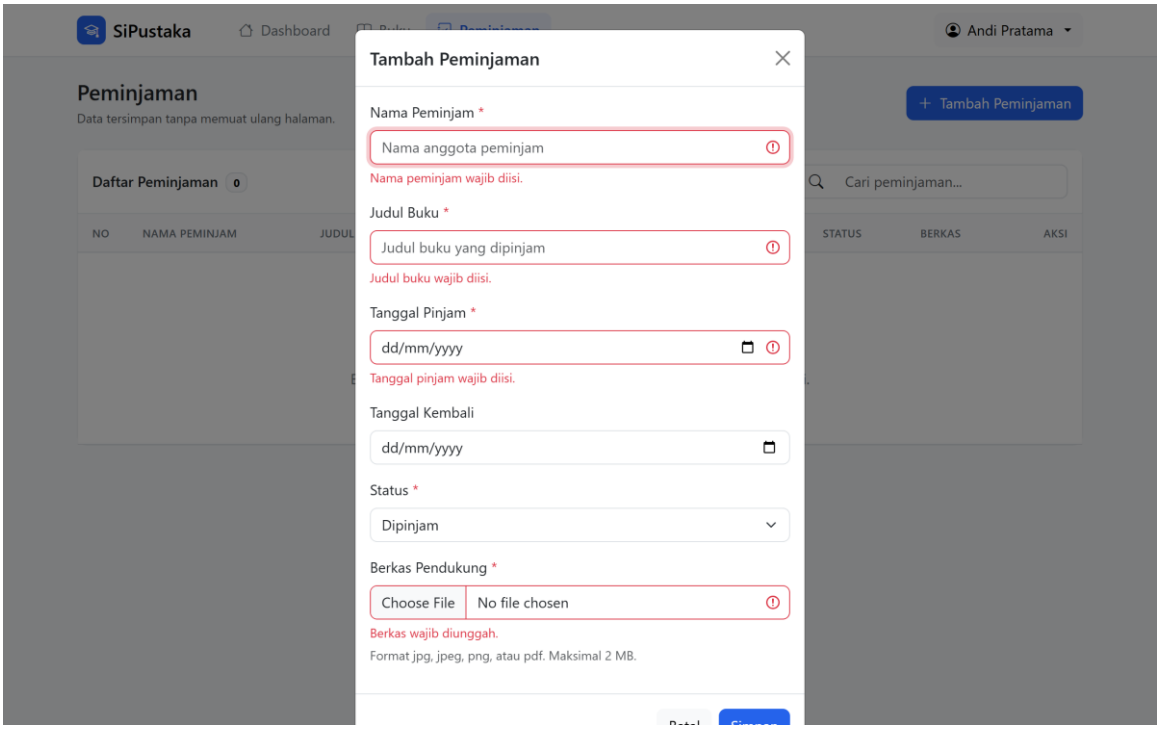


Gambar 25. Halaman peminjaman saat belum ada data.



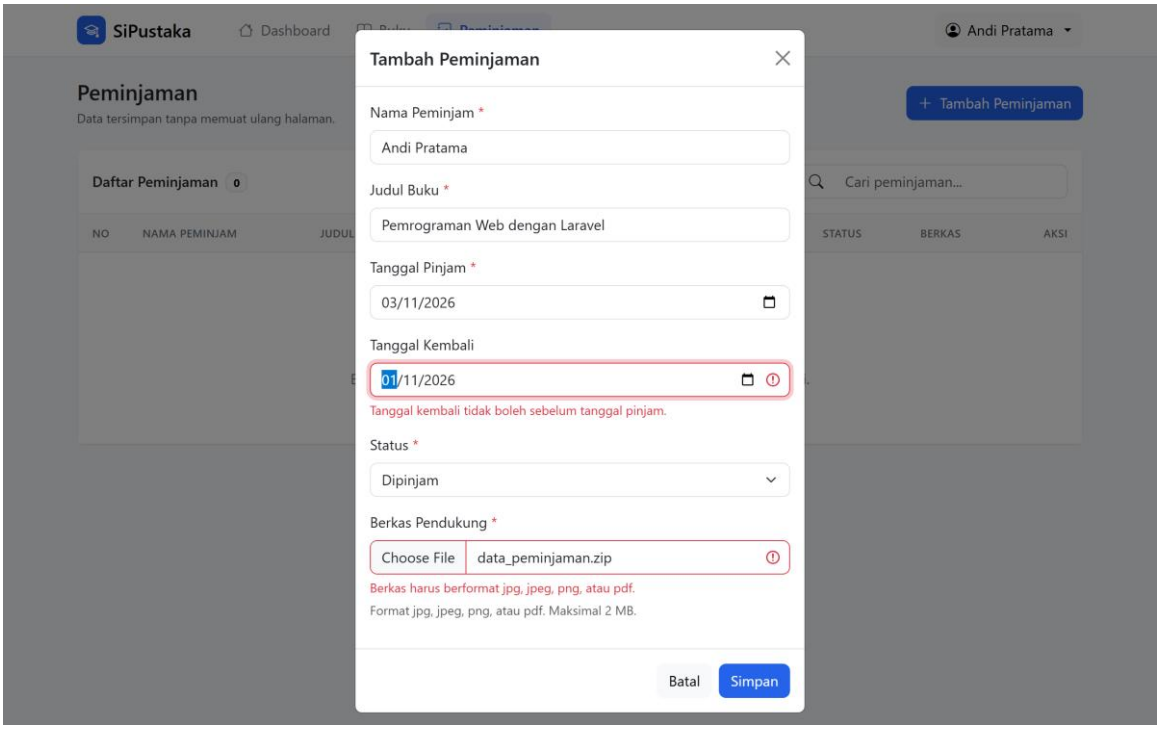
Gambar 26. Modal tambah peminjaman.

Tombol Simpan pada form kosong dibalas 422 dan pesan muncul di bawah tiap input, termasuk input berkas, tanpa halaman dimuat ulang.

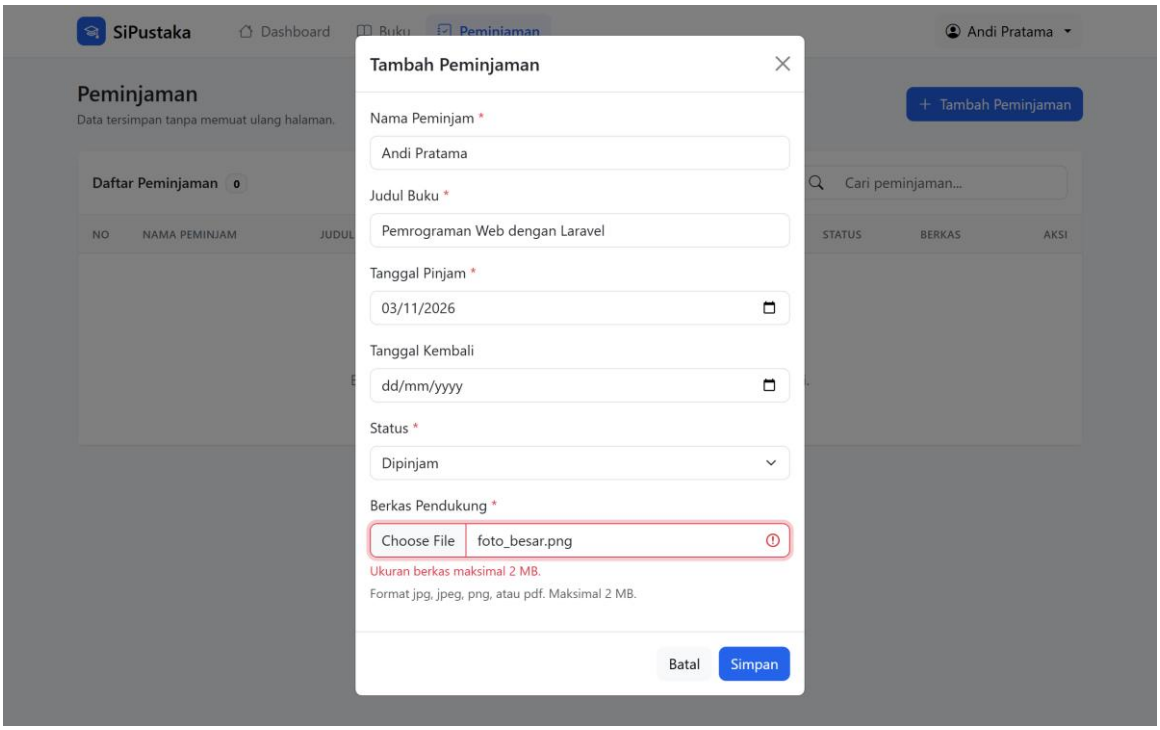


Gambar 27. Error 422 saat form dikirim kosong.

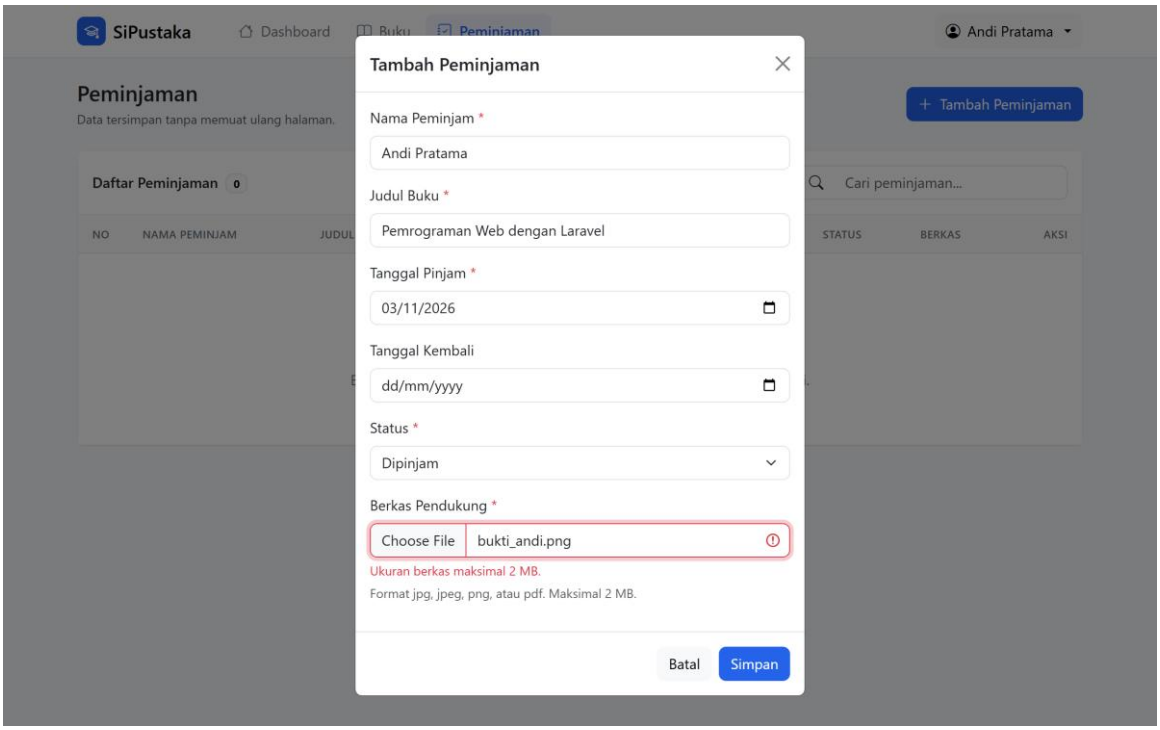
Berkas berformat zip ditolak dengan pesan format. Pada gambar yang sama, tanggal kembali yang lebih awal dari tanggal pinjam juga ditolak.



Gambar 28. Error 422 untuk berkas zip dan tanggal kembali yang tidak valid.

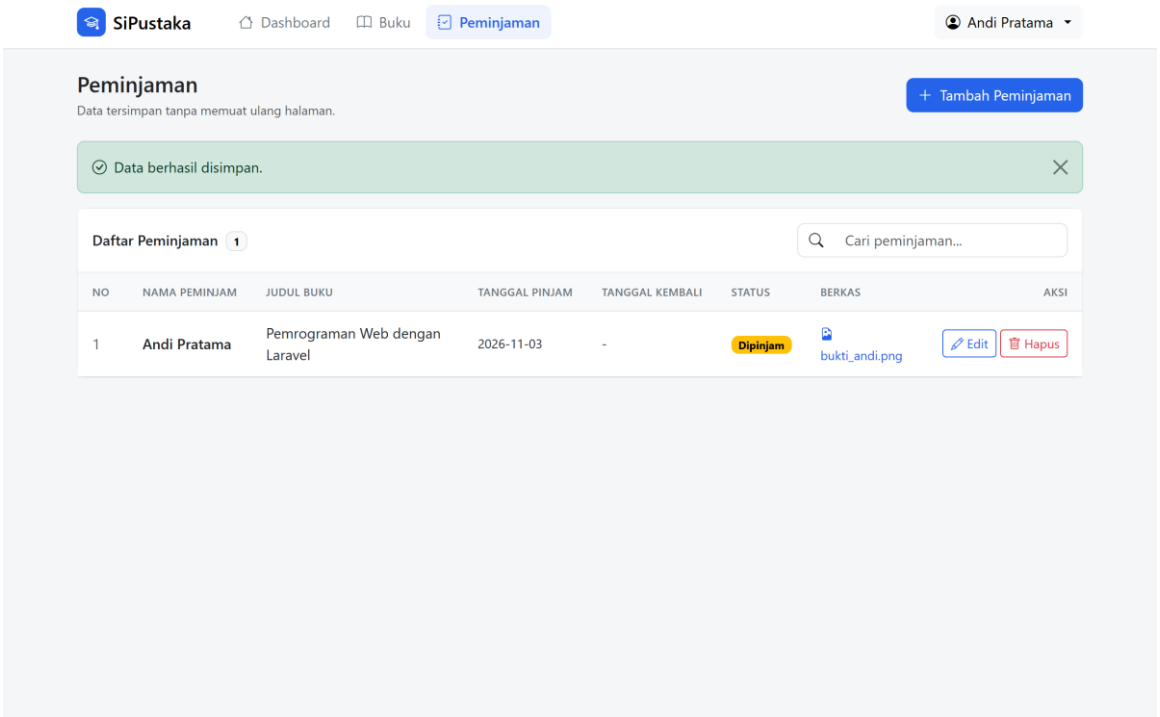


Gambar 29. Error 422 untuk berkas yang melebihi 2 MB.

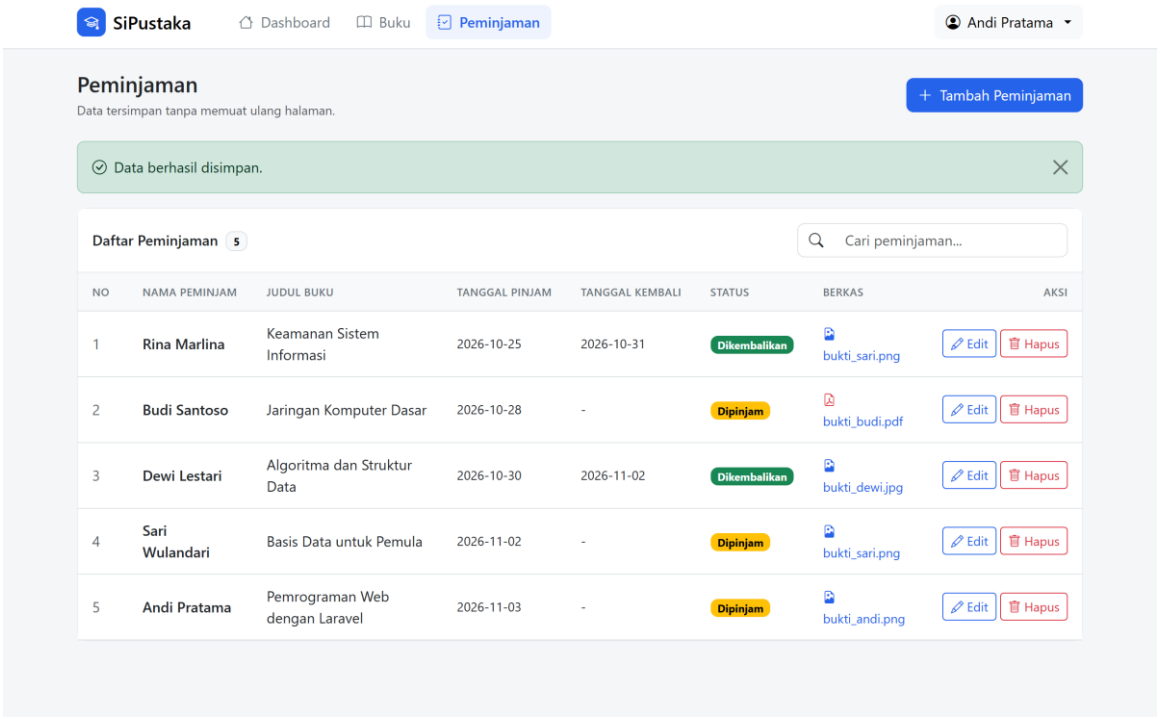


Gambar 30. Modal yang sudah diisi dengan berkas pendukung.

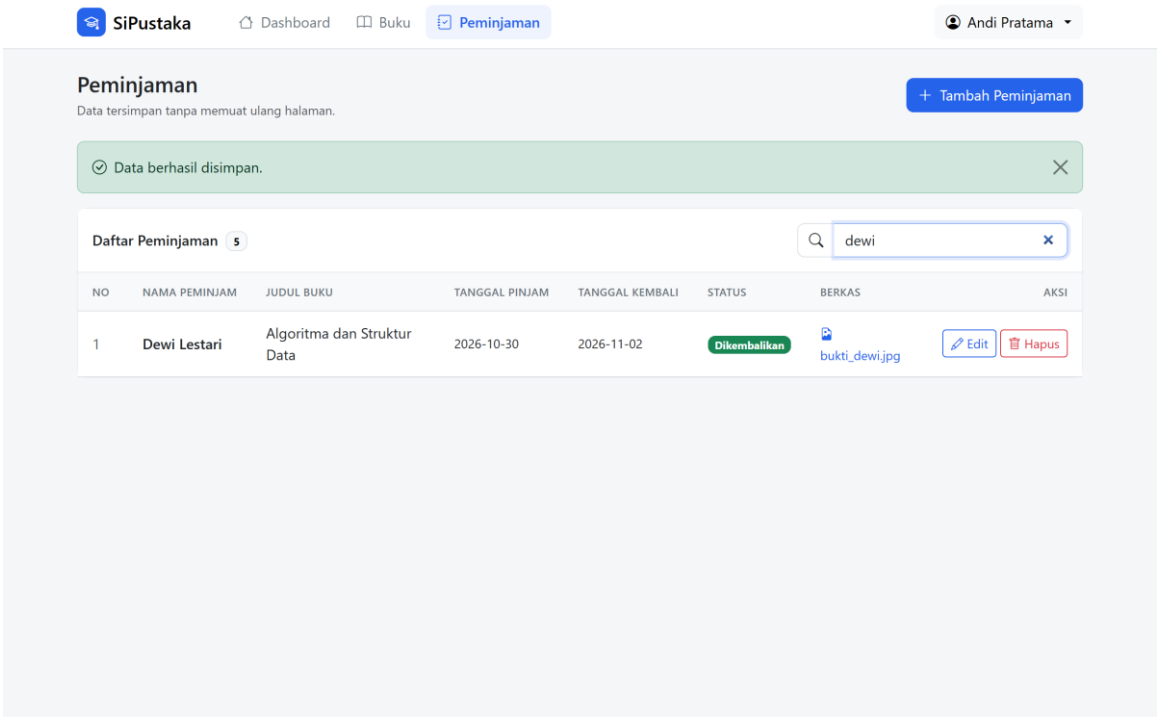
Setelah berhasil, modal menutup dan tabel diperbarui bersama pesan sukses di atas tabel, tanpa muat ulang halaman. Tautan pada kolom Berkas membuka berkas yang tadi diunggah.



Gambar 31. Pesan berhasil setelah data disimpan lewat AJAX.

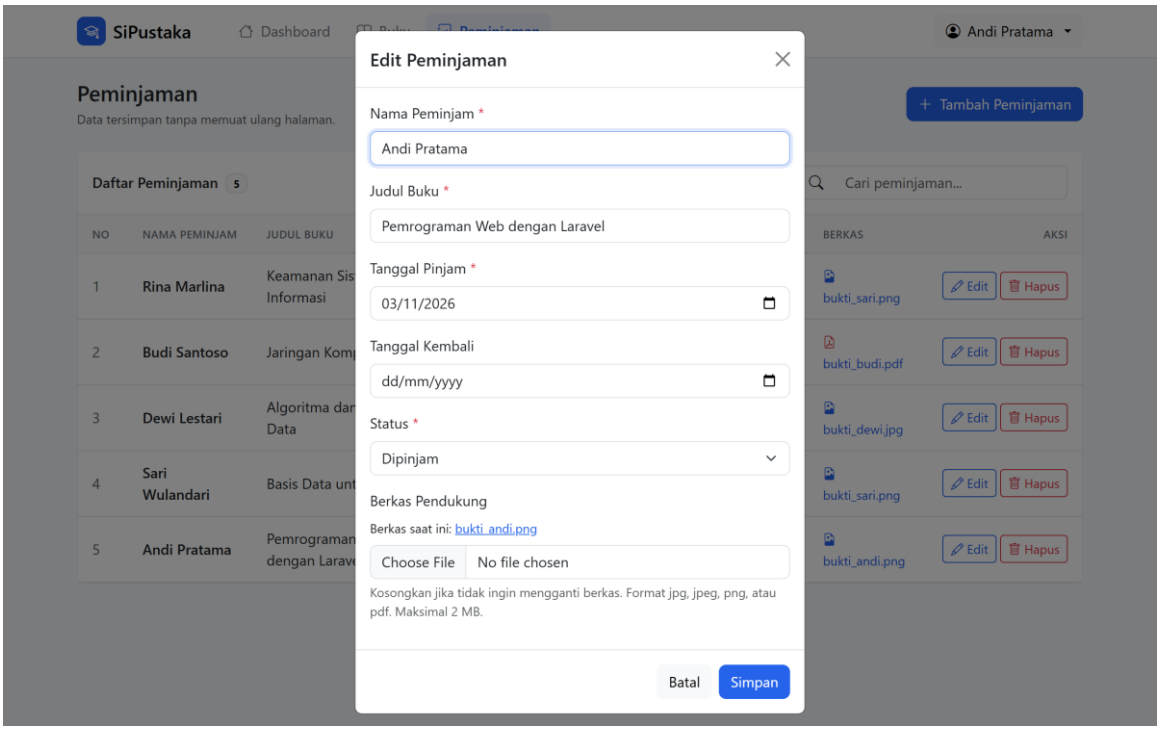


Gambar 32. Tabel peminjaman dengan beberapa data.



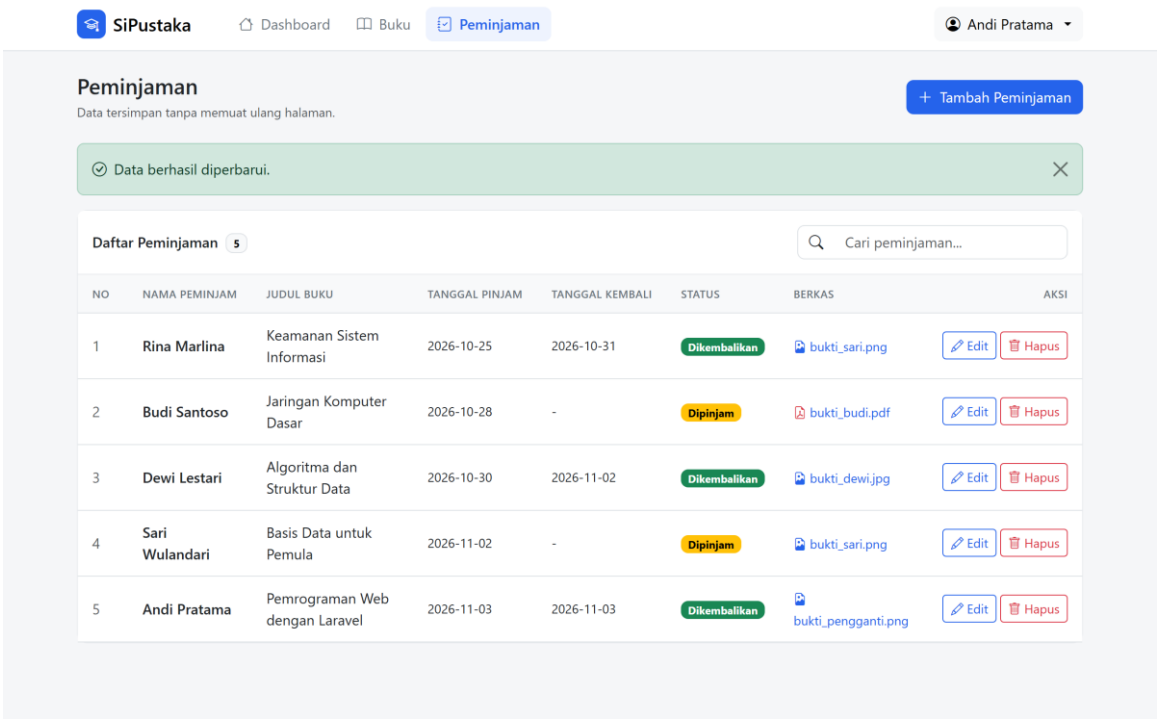
Gambar 33. Pencarian pada tabel peminjaman.

Modal edit terisi dari data server dan menampilkan tautan berkas yang sedang tersimpan. Input berkas boleh dikosongkan bila berkas tidak ingin diganti.

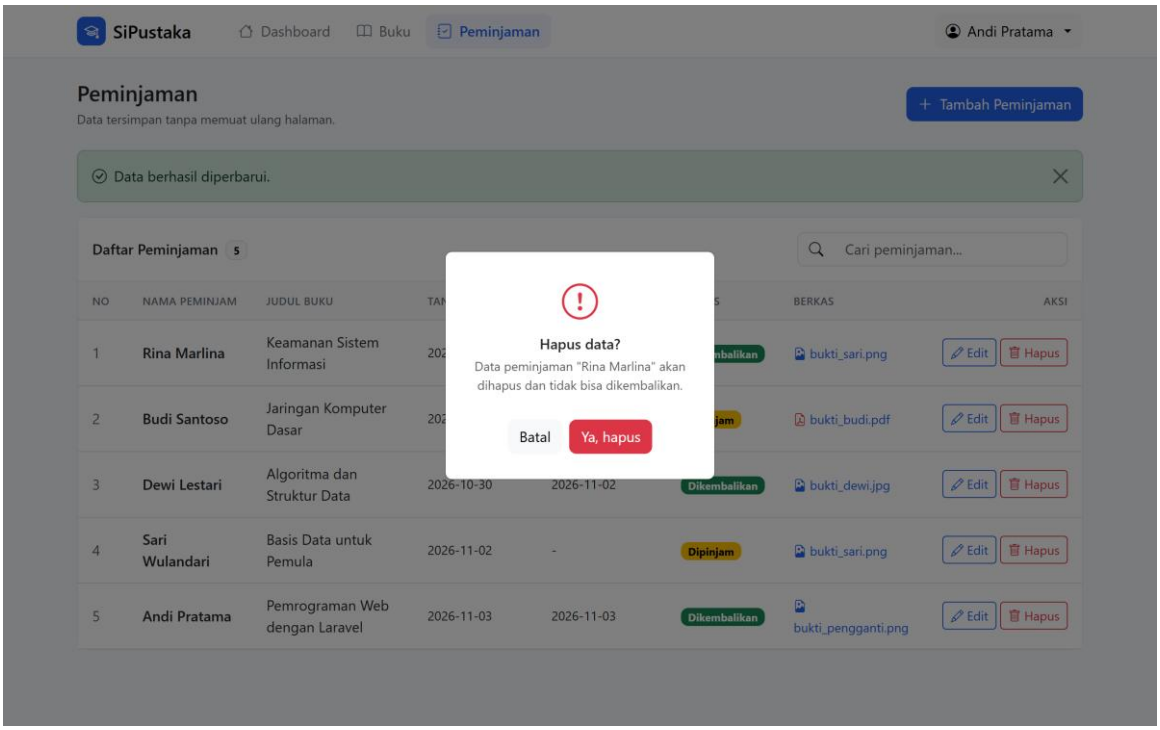


Gambar 34. Modal edit peminjaman.

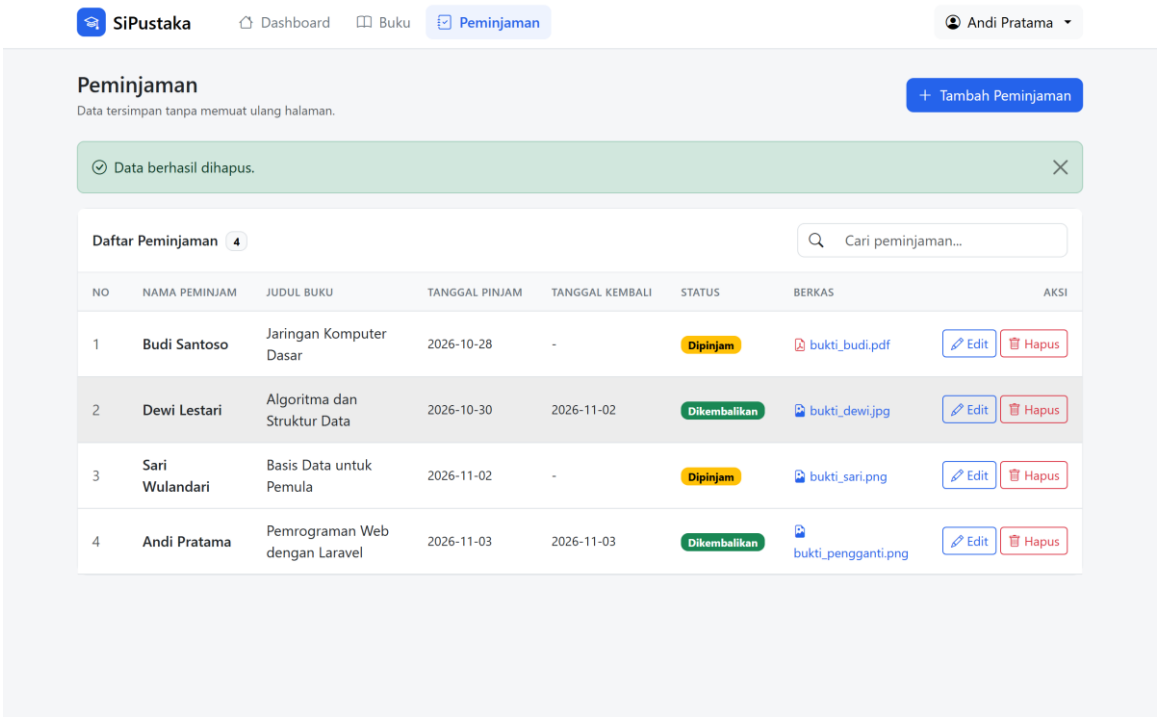
Pada data Andi Pratama saya mengubah status menjadi Dikembalikan sekaligus mengganti berkas. Nama berkas pada tabel berubah dari bukti_andi.png menjadi bukti_pengganti.png.



Gambar 35. Pesan berhasil setelah data diperbarui lewat AJAX.

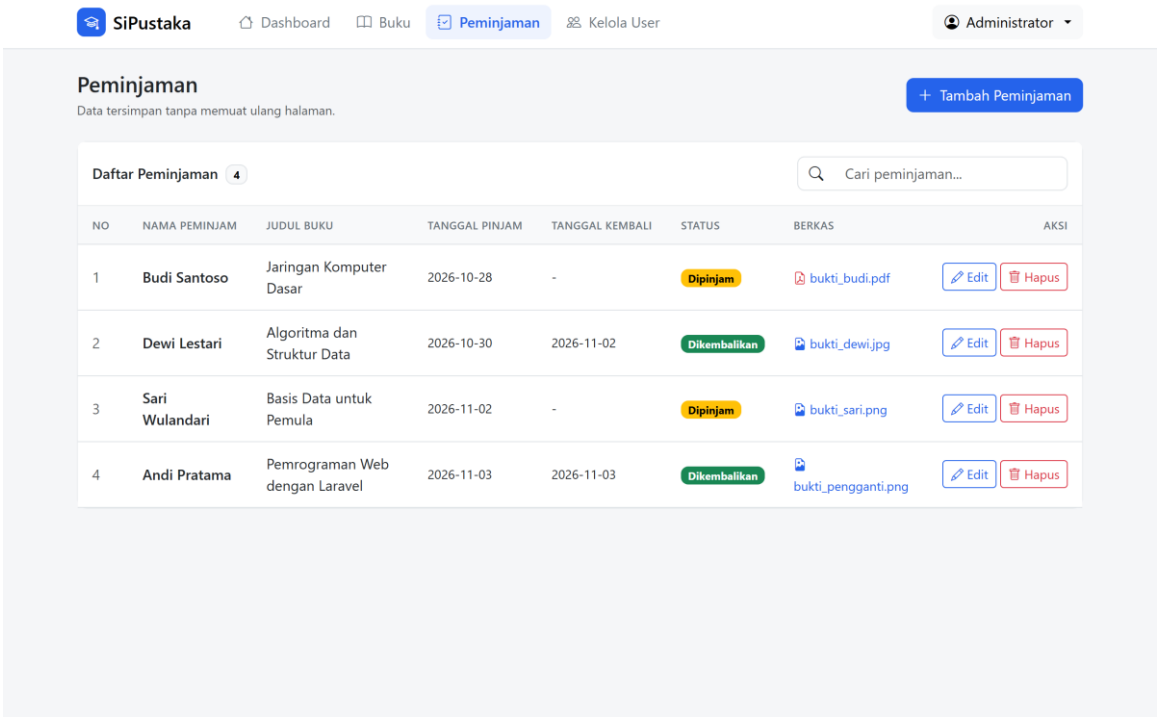


Gambar 36. Modal konfirmasi sebelum peminjaman dihapus.



Gambar 37. Pesan berhasil setelah peminjaman dihapus lewat AJAX.

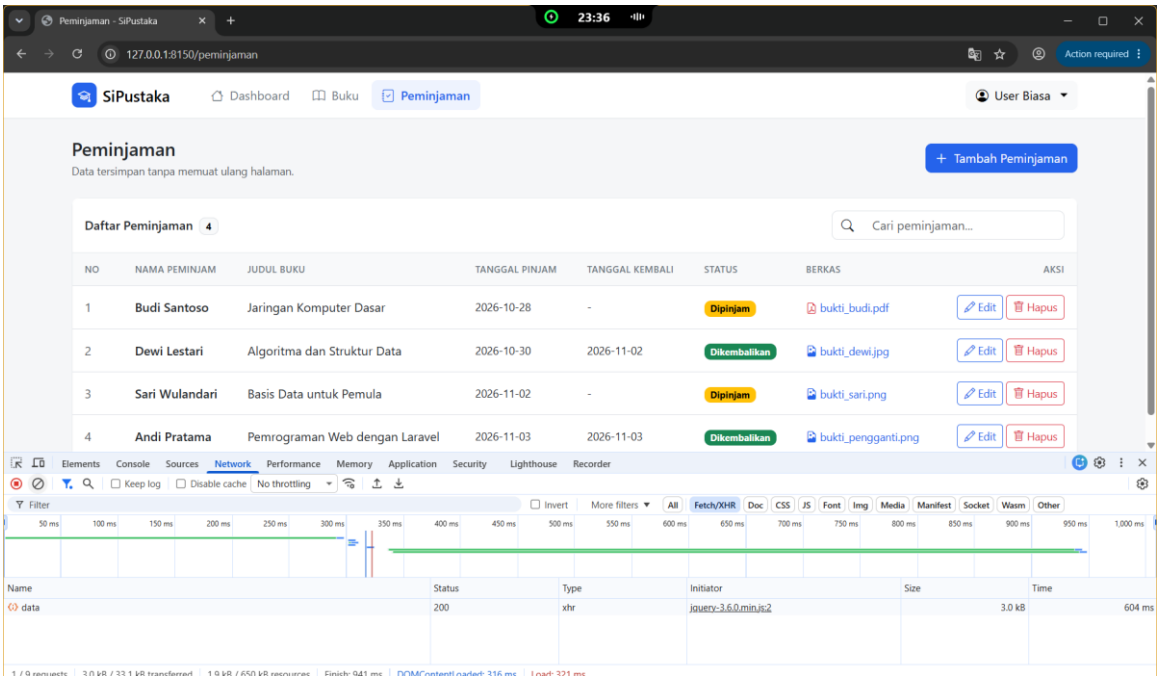
Akun admin juga dapat membuka halaman yang sama dan melihat seluruh data.



Gambar 38. Halaman peminjaman pada akun admin.

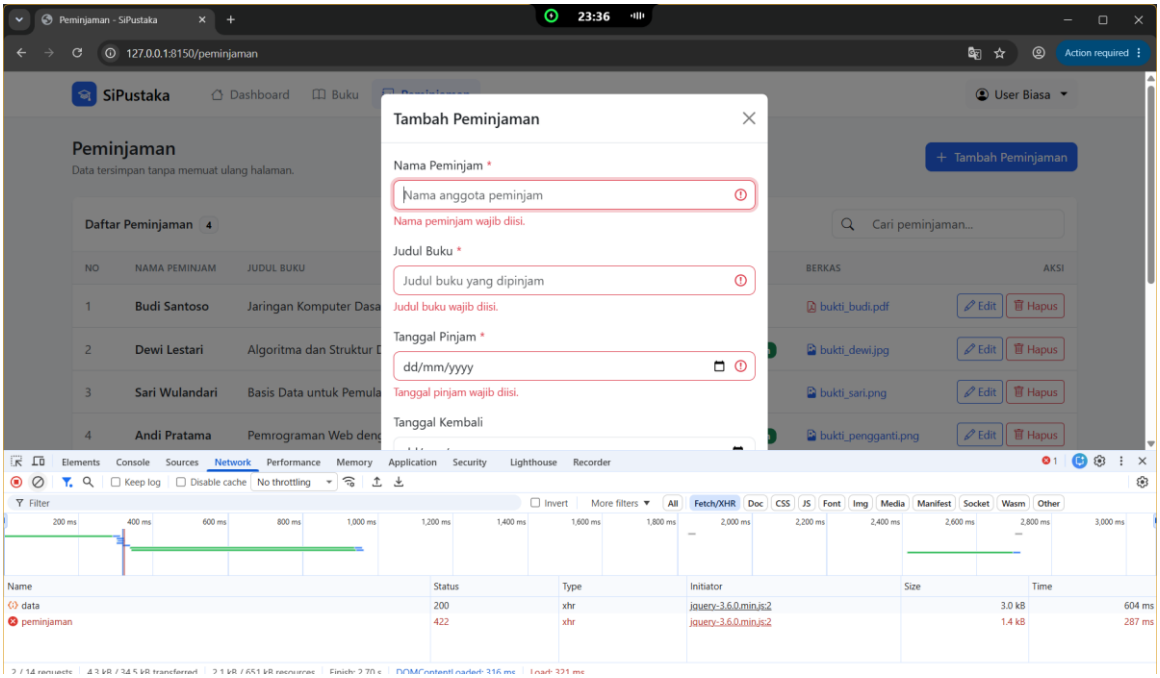
D.6 Bukti AJAX pada DevTools

Tab Network pada Chrome DevTools saya filter ke Fetch/XHR. Ketika halaman dibuka, hanya ada satu request xhr bernama data dengan status 200. Inisiatornya jquery, tanda bahwa tabel diisi lewat AJAX.



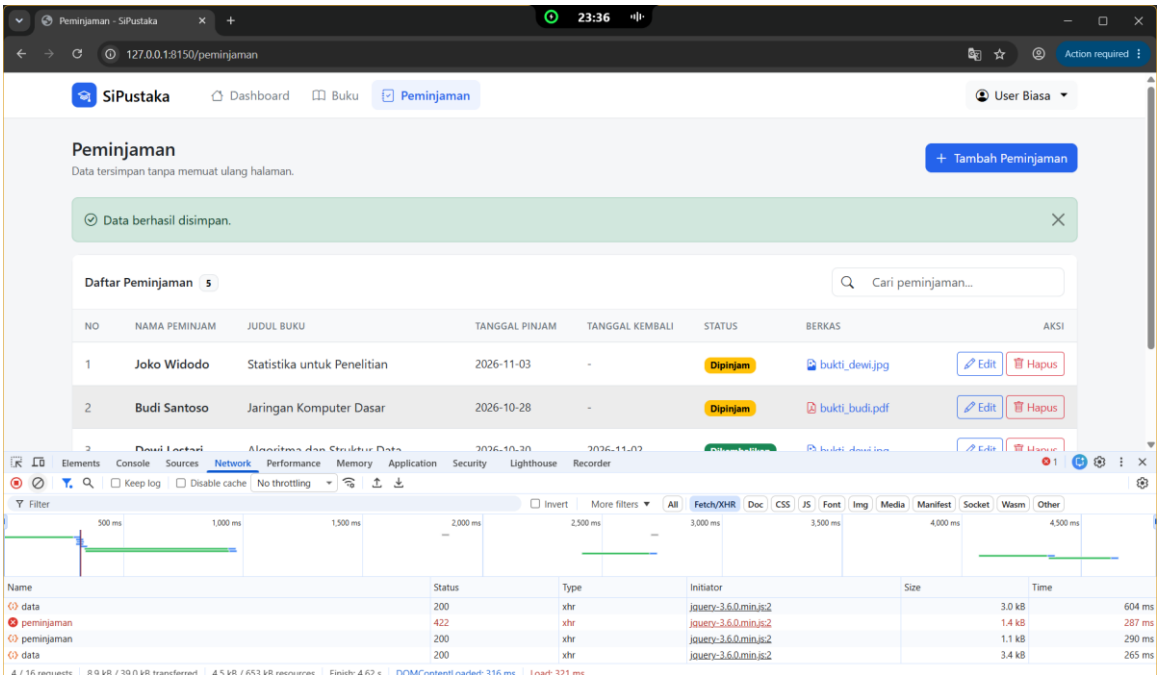
Gambar 39. Request data (GET, xhr) saat halaman peminjaman dimuat.

Form yang dikirim kosong menghasilkan request peminjaman dengan status 422.



Gambar 40. Request peminjaman berstatus 422 pada tab Network.

Setelah form diisi dengan benar, muncul request peminjaman berstatus 200 diikuti request data untuk memuat ulang isi tabel. Data pada tiga tangkapan DevTools berasal dari sesi uji terpisah, sehingga isi tabelnya berbeda dari tangkapan layar lainnya.



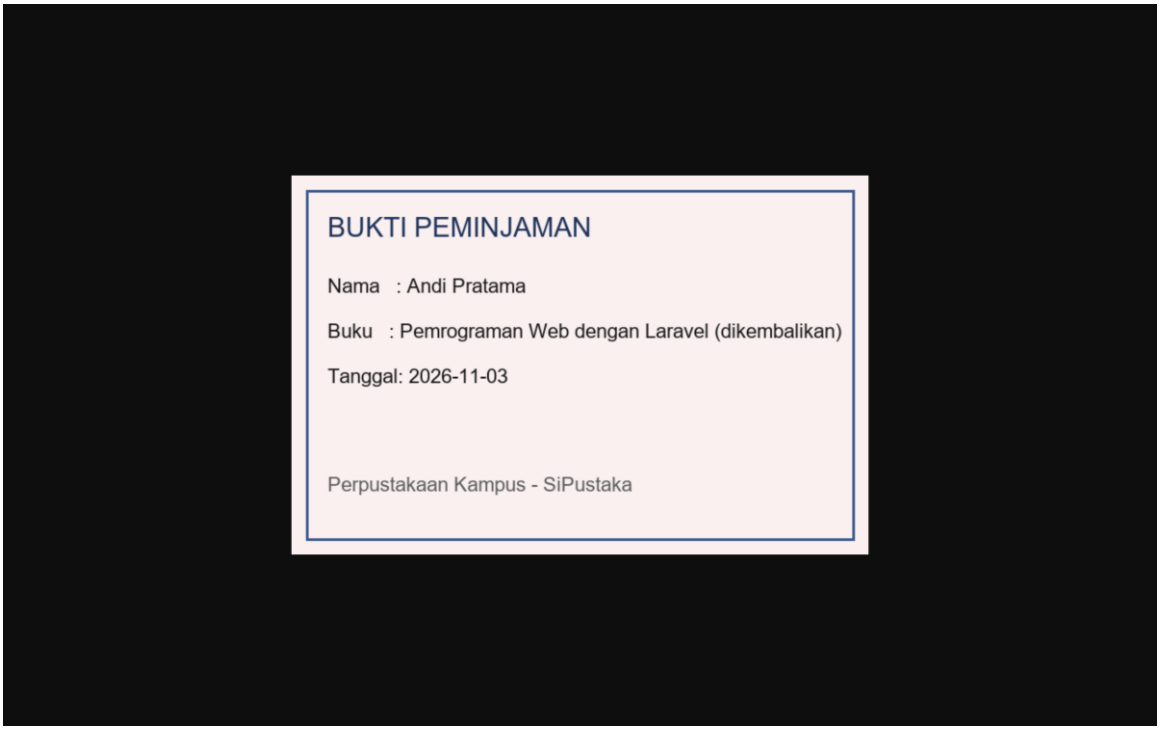
Gambar 41. Urutan request data, peminjaman 422, peminjaman 200, dan data.

D.7 Berkas, Storage, dan Database

Endpoint peminjaman/data mengembalikan array JSON berisi semua peminjaman beserta berkas_url. Inilah yang dibaca jQuery untuk mengisi tabel.

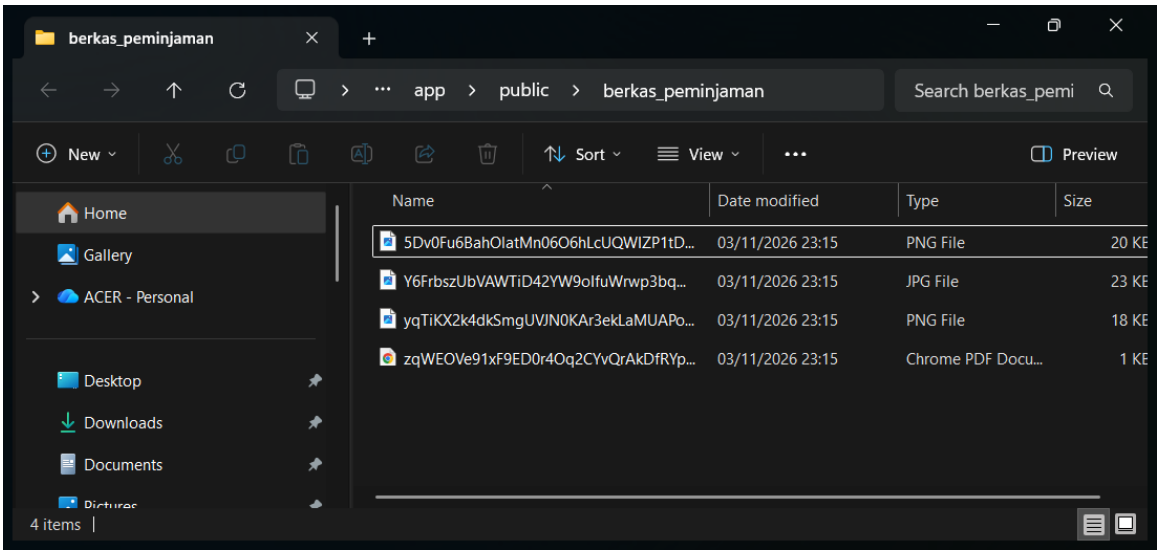


Gambar 42. Respons JSON dari peminjaman/data.



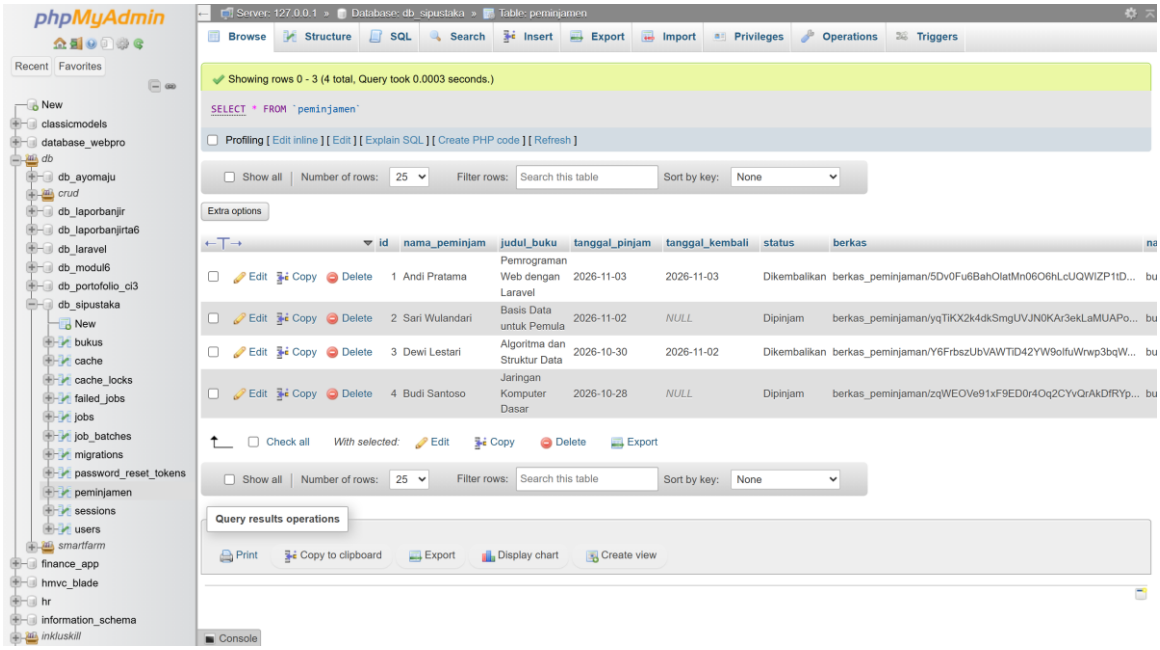
Gambar 43. Berkas yang dibuka lewat tautan pada kolom Berkas.

Isi folder storage/app/public/berkas_peminjaman berjumlah empat berkas dan cocok dengan empat baris data. Nama berkas di disk berupa hash acak dari Laravel.

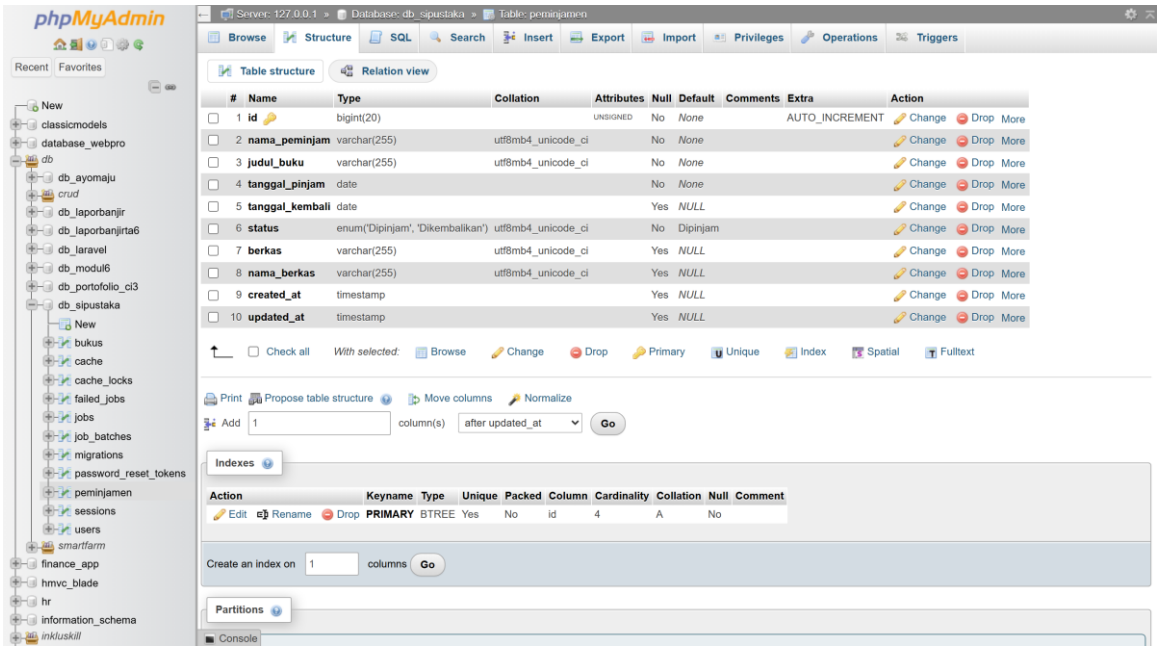


Gambar 44. Isi folder storage/app/public/berkas_peminjaman.

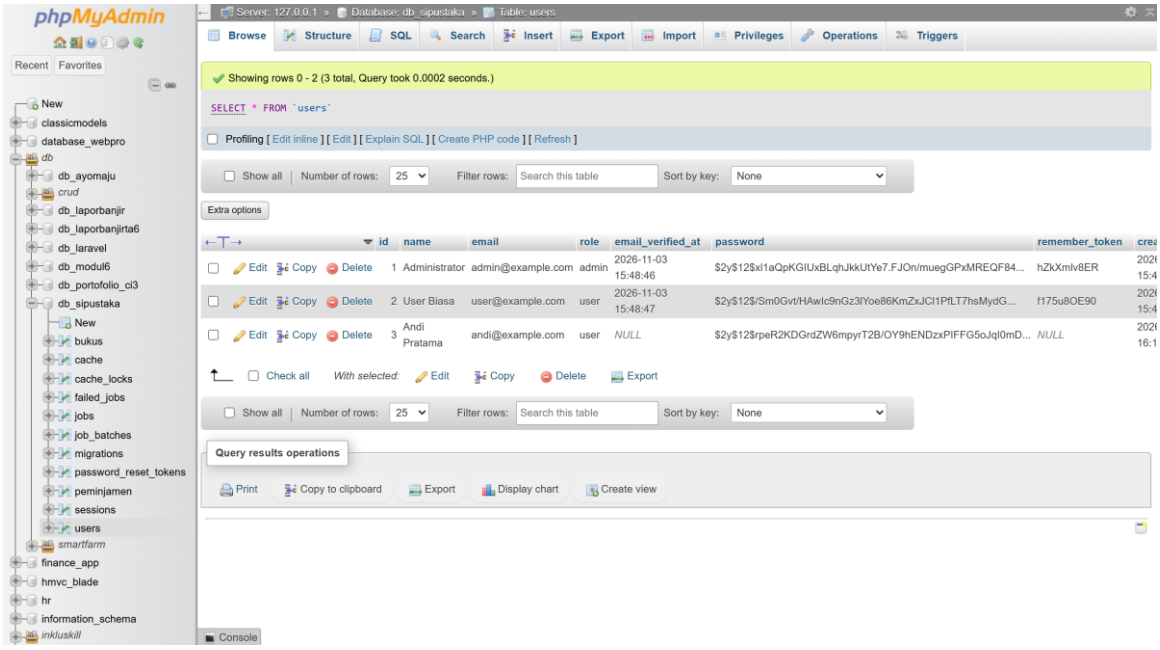
Di phpMyAdmin, kolom berkas pada tabel peminjamen menyimpan path relatif seperti berkas_peminjaman/xxx.png, bukan isi berkas.



Gambar 45. Isi tabel peminjamen pada phpMyAdmin.



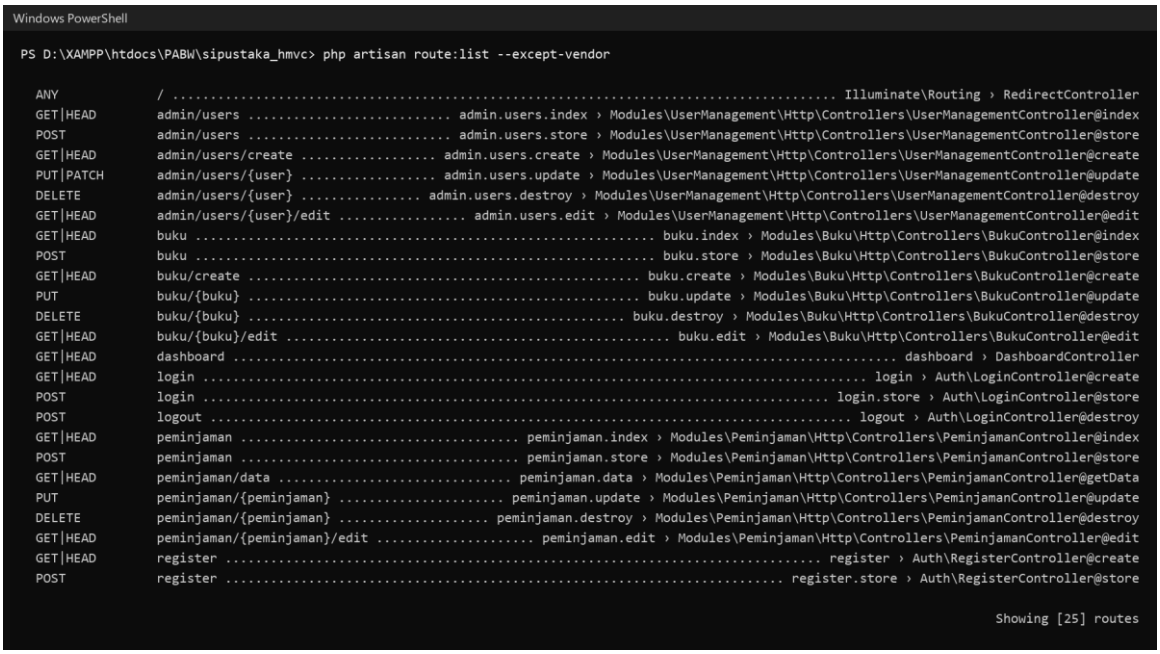
Gambar 46. Struktur tabel peminjamen pada phpMyAdmin.



Gambar 47. Isi tabel users dengan kolom role pada phpMyAdmin.

D.8 Daftar Route

Perintah route:list --except-vendor menampilkan 25 route. Route buku, peminjaman, dan admin/users tercatat dengan controller dari folder Modules, sedangkan login, register, logout, dan dashboard berasal dari app.



Gambar 48. Hasil php artisan route:list --except-vendor.

E. Analisis dan Pembahasan

E.1 Cara Kerja HMVC

HMVC membagi aplikasi menjadi modul. Tiap modul punya controller, model, view, request, route, dan migration sendiri di dalam folder Modules. Pada project ini modul Buku, Peminjaman, dan UserManagement berdiri sendiri, sedangkan autentikasi tetap di app karena dipakai oleh semuanya. Namespace mengikuti nama modul, misalnya Modules\Peminjaman\Http\Controllers\PeminjamanController.

Blok merge-plugin membuat Composer membaca composer.json setiap modul saat dump-autoload. Tanpa itu, kelas modul tidak dikenali walau file-nya ada. Route modul juga tidak perlu ditulis di routes/web.php karena service provider modul memuat routes/web.php miliknya sendiri. Hal ini terlihat pada route:list.

E.2 Authentication dan Authorization

Authentication menjawab siapa penggunanya, authorization menjawab apa yang boleh dilakukannya. Di aplikasi ini authentication ditangani LoginController dan RegisterController melalui Auth::attempt dan Auth::login. Authorization ditangani middleware EnsureUserHasRole yang membaca kolom role.

Keduanya bisa dibedakan dari hasilnya. Pengunjung yang belum login dialihkan ke halaman login oleh middleware auth. Pengguna yang sudah login tetapi rolenya tidak cukup mendapat

403 dari middleware role. User yang sudah login bisa membuka peminjaman, tetapi ditolak saat membuka /admin/users.

E.3 Cara Kerja Middleware Role

Pada role:admin atau role:admin,user, bagian setelah titik dua dikirim Laravel sebagai argumen tambahan ke method handle(). Middleware lalu memanggil hasRole() pada user yang sedang login. Jika tidak cocok, abort(403) dijalankan dan request tidak pernah sampai ke controller.

Akun hasil register selalu menjadi user karena role diisi di RegisterController, tidak diambil dari input. Untuk admin, batasan akun sendiri dipasang di dua tempat. UpdateUserRequest mencegah penurunan role dan method destroy menolak penghapusan.

E.4 Perbedaan CRUD Synchronous dan AJAX

Aspek	**Synchronous (Buku, UserManagement)**	**AJAX (Peminjaman)**
Respons controller	view() atau redirect()->with(...)	response()->json([...])
Perilaku halaman	Dimuat ulang setiap aksi	Tidak dimuat ulang, tabel diperbarui JavaScript
Pesan berhasil	Session flash	Alert dibuat JavaScript dari JSON
Error validasi	Redirect kembali dan @error di Blade	Status 422 dan JSON errors ditampilkan jQuery
Token CSRF	@csrf di dalam form	Header X-CSRF-TOKEN lewat $.ajaxSetup
Form tambah dan edit	Halaman terpisah	Satu modal Bootstrap
Upload berkas	Form multipart biasa	FormData, processData false, contentType false

Synchronous lebih sederhana dan mudah di-debug, serta tetap jalan tanpa JavaScript. AJAX terasa lebih cepat karena posisi halaman dan isi kotak pencarian tidak hilang, tetapi JavaScript harus menangani sendiri status 422 dan memasang pesan error pada setiap input.

E.5 FormData dan Penyimpanan Berkas

serialize() hanya menghasilkan pasangan teks, jadi isi file tidak ikut. FormData membaca semua input termasuk file, sehingga satu request membawa data teks dan berkas sekaligus. processData false mencegah jQuery mengubah FormData menjadi query string, dan contentType false membiarkan browser menetapkan header multipart beserta boundary-nya. Karena file hanya bisa dikirim lewat POST, request edit memakai POST dengan field _method bernilai PUT.

Berkas disimpan di storage dan database hanya menyimpan path serta nama aslinya. Tabel tetap ringan, berkas bisa disajikan langsung lewat link storage, dan berkas bisa diganti tanpa mengubah struktur tabel. Nama berkas di disk dibuat acak sehingga dua berkas bernama sama tidak saling menimpa.

Pada edit tanpa berkas baru, berkas lama dibiarkan. Pada edit dengan berkas baru, berkas lama dihapus lebih dulu. Pada hapus data, berkas dihapus sebelum barisnya. Urutan ini mencegah berkas yatim di storage, dan pada pemeriksaan terakhir folder storage berisi empat berkas, sama dengan jumlah baris tabel.

Request tanpa token CSRF akan dibalas 419 oleh Laravel. Hal itu tidak terjadi pada praktikum ini karena meta csrf-token dan header X-CSRF-TOKEN sudah dipasang sejak awal.

E.6 Kendala dan Penyelesaian

1. Tinker menampilkan ParseErrorException: unexpected T_OBJECT_OPERATOR on line 1. Kode PHP yang saya kirim ke Tinker lewat perintah satu baris di PowerShell gagal di-parse oleh PsySH pada baris pertama. Pemeriksaan data tidak saya lanjutkan lewat Tinker, tetapi lewat phpMyAdmin dan skrip PHP terpisah.

2. php artisan db:show dan about menghasilkan SQLSTATE[42S02] 1146 Table 'performance_schema.session_status' doesn't exist. Kedua perintah itu membaca tabel performance_schema yang tidak ada pada MariaDB bawaan XAMPP. Saya tidak memakainya lagi dan memeriksa isi database lewat phpMyAdmin.

3. route:list --columns gagal dengan pesan option does not exist karena versi Laravel ini tidak punya opsi tersebut. Saya memakai route:list --except-vendor.

4. Teks bantuan pada form user sempat memuat kata Pendaftaran, sisa dari project latihan sebelumnya. Teks itu saya ganti menjadi User hanya dapat melihat daftar buku dan mengelola peminjaman.

5. Form dengan atribut required membuat browser menahan pengiriman dan memunculkan pesan bawaan Please fill out this field, sehingga pesan dari Form Request tidak tampil. Untuk tangkapan layar error server, validasi HTML5 saya matikan lewat properti noValidate pada form, tanpa mengubah kode aplikasi.

E.7 Catatan Perbedaan dengan Modul

1. Pada tabel perbandingan di modul, modul synchronous disebut UserManagement dan modul AJAX disebut Pendaftaran. Pada tugas SiPustaka, modul AJAX adalah Peminjaman, sedangkan CRUD synchronous terbagi dua, yaitu Buku dan UserManagement.

2. Teks pagination bawaan Bootstrap pada daftar buku masih berbahasa Inggris, misalnya Showing 1 to 10 of 13 results, karena view pagination bawaan Laravel tidak saya ubah.

3. Tangkapan DevTools diambil dari sesi uji terpisah, sehingga datanya, termasuk satu nama yang tidak muncul pada tangkapan lain, berbeda dari data akhir.

4. Bukti penghapusan akun sendiri memakai form DELETE yang saya sisipkan lewat JavaScript, karena tombolnya memang disembunyikan.

5. Server dijalankan pada port 8150 karena port 8000 sedang dipakai proses lain.

F. Kesimpulan

SiPustaka berhasil dibangun dengan arsitektur HMVC yang terdiri dari modul Buku, Peminjaman, dan UserManagement. Autentikasi yang saya tulis sendiri di luar modul bekerja bersama middleware role, sehingga halaman khusus admin menampilkan 403 untuk user dan akun hasil register selalu menjadi user. Admin pun tidak dapat menghapus atau menurunkan role akunnya sendiri.

CRUD Buku dan UserManagement berjalan secara synchronous dengan pesan sukses dan error di bawah setiap input. CRUD Peminjaman berjalan dengan AJAX, menampilkan error 422 di bawah input, dan memperbarui tabel tanpa muat ulang. Berkas peminjaman tersimpan di storage, bisa dibuka lewat tautan, diganti saat edit, dan ikut terhapus saat datanya dihapus. Tab Network pada DevTools mengonfirmasi bahwa modul ini memakai request xhr.

Dari praktikum ini saya jadi paham bahwa authentication dan authorization adalah dua lapis yang berbeda, bahwa pembatasan akses harus ada di route dan tidak cukup dengan menyembunyikan tombol, dan bahwa AJAX menuntut penanganan status dan pesan error yang lebih teliti di sisi klien.

G. Link Repository Github

https://github.com/Dryhus21/PABW.git